

ABDUCTIONBENCH: BENCHMARKING ABDUCTIVE REASONING IN LARGE LANGUAGE MODELS

Anonymous authors

Paper under double-blind review

ABSTRACT

Abductive reasoning is the inference of plausible explanations for observed evidence, particularly when information is incomplete or supports multiple competing hypotheses. Evaluating this capability is not a single task: a system may need to generate an explanation, discriminate among competing hypotheses, revise its hypotheses as evidence arrives, or decide what evidence to seek next. Yet these capabilities are typically evaluated through isolated benchmarks with incompatible task formulations and scoring procedures, so claims that a model is “good at abduction” may reflect only a narrow slice of the capability. We introduce ABDUCTIONBENCH, a unified framework that turns this fragmented landscape into a common evaluation space. Its taxonomy characterizes abductive tasks by hypothesis format, application domain, and evidence-delivery protocol, covering generation and selection tasks across static, passive, and interactive settings while preserving their native execution structure. We instantiate this framework on 27 datasets (21 static, 5 interactive, 1 passive) and evaluate eight LLMs from four model series—GPT, Gemini, Gemma, and Qwen—together with Jev, a non-LLM System One decision system evaluated on the single-choice selection tasks. Beyond harmonizing heterogeneous tasks, AbductionBench evaluates final-answer quality separately from the observable reasoning traces and interaction histories produced alongside it. We analyze these written traces as observable model outputs rather than as faithful representations of internal computation, characterizing evidence use, hypothesis comparison, uncertainty, contradiction handling, and answer commitment. Chain-of-thought is not uniformly beneficial: it helps logical and diagnostic tasks but often hurts open-ended hypothesis generation, and within a dataset, longer traces go with lower accuracy. In interactive tasks, the relevance of the model’s questions falls as an episode proceeds, and a relevant first question predicts a correct final answer; in the passive task, models rarely revise their first hypothesis as new evidence arrives. AbductionBench therefore shifts abductive evaluation from reporting performance on disconnected datasets toward diagnosing the conditions under which models succeed or fail at abductive tasks.

1 INTRODUCTION

Abductive reasoning seeks plausible explanations for observed evidence. When information is incomplete, several hypotheses may fit the same observations, and new evidence may change which explanation is best supported. For large language models (LLMs), a correct final answer alone does not show whether the model used the relevant evidence, considered alternatives, or revised an earlier hypothesis. Evaluation therefore needs to examine outcomes alongside observable reasoning and evidence-gathering behavior, without treating written traces as direct records of internal computation.

Existing benchmarks address parts of this problem. Abductive Commonsense Reasoning evaluates narrative explanation selection and generation (Bhagavatula et al., 2020); GEAR evaluates generated hypothesis sets on four programmable benchmarks (He et al., 2025). Because each benchmark fixes its own formulation and scoring, results do not show how

abductive ability varies with the kind of hypothesis required, the domain, or the way evidence arrives. Most benchmarks also supply all evidence at once, whereas practical abduction—clinical diagnosis, incident response, threat attribution—requires revising a hypothesis as evidence arrives or deciding which evidence to seek.

AbductionBench addresses this gap. It brings existing tasks into one execution and reporting protocol and describes each along three axes: hypothesis generation or selection, application domain, and evidence delivery, which is *static* (all evidence at the outset), *passive* (evidence arrives in a fixed order and the model revises its hypothesis after each turn), or *interactive* (the model’s questions or actions determine what it observes next). It scores final answers with task-appropriate checks or an LLM judge, and separately analyzes chain-of-thought (CoT) traces for evidence use, hypothesis comparison, uncertainty, and commitment, and multi-turn episodes for question relevance and hypothesis revision.

We evaluate eight LLMs from four model families on 27 datasets (21 static, five interactive, one passive), together with Jev, a non-LLM System One decision system, on the single-choice tasks. CoT helps when an explanation must be derived by chaining evidence but often hurts open-ended hypothesis generation, and within a dataset, longer traces go with lower accuracy. Jev is competitive with small LLMs when selecting among explanations over compact evidence but falls behind on long cases. In interactive diagnosis, a relevant first question goes with a correct final answer and relevance declines as the interaction proceeds; with passively arriving evidence, models rarely revise their first hypothesis.

Our contributions are:

- **A structured suite** of 27 datasets spanning ten domains, hypothesis generation and selection, and three modes of evidence delivery (static, passive, interactive), including a passive adaptation of AthenaBench, with source-task adaptations documented.
- **A complementary evaluation protocol** that reports final-answer outcomes alongside trace measures for static tasks and turn-level measures for passive and interactive episodes.
- **An empirical study** of eight LLMs and a non-LLM System One decision system (Jev), covering 69,599 static responses and 2,400 passive and interactive episodes.

2 RELATED WORK

Abduction seeks plausible explanations for observations rather than conclusions that must be true (Schurz, 2008). Prior benchmarks isolate individual pieces of this problem: ART’s α NLI asks models to select the more plausible explanation connecting two observations, while e-CARE includes both a two-choice causal-reasoning task and an explanation-generation task (Bhagavatula et al., 2020; Du et al., 2022). α NLG and UNcommonsense focus on open-ended explanation generation, whereas True Detective and MuSR evaluate answer selection in longer narratives (Zhao et al., 2024; Del & Fishel, 2023; Sprague et al., 2024). ProofWriter and AbductionRules study abduction over natural-language rule bases, and HypoSpace evaluates set-valued hypothesis generation under underdetermination (Tafford et al., 2021; Young et al., 2022; Chen et al., 2026). In clinical settings, MedCaseReasoning evaluates diagnosis together with alignment to clinician-authored reasoning, while DDXPlus supports interactive evidence collection and differential diagnosis (Wu et al., 2025; Tchango et al., 2022). On the evaluation side, IBE-Eval scores explanations by consistency, parsimony, coherence, and uncertainty (Dalal et al., 2024), and RECV reports higher error on claim verification that requires abduction from incomplete evidence (Dougrez-Lewis et al., 2025).

Closest to our work, GEAR (He et al., 2025) evaluates sets of executable hypotheses by consistency, generalizability, and diversity, but is restricted to domains where hypotheses compile to code and has no candidate-selection task, so it cannot separate generating good candidates from discriminating among them. *Wiring the “Why”* (Salimi et al., 2026a) surveys over 60 papers and compares generation and selection empirically, but with a fixed prompt per benchmark, a small set of models, and static benchmarks only. AbductionBench evaluates generation and selection as distinct tasks across 27 datasets and ten domains,

under static, passive, and interactive evidence delivery, and reports reasoning-trace and interaction measures alongside outcome accuracy.

3 BENCHMARK SCOPE AND DESIGN

AbductionBench brings existing tasks into a common evaluation interface. This section defines which tasks fall within its scope, how they are categorized, and how datasets are integrated; execution and scoring are specified in Section 4.

3.1 WHAT COUNTS AS AN ABDUCTIVE TASK

We include benchmarks designed to evaluate abductive reasoning and selected tasks from other benchmarks. For each task, we identify what is given, what the model must infer, and why that inference is abductive: usually, the model infers a hypothesis that explains supplied observations. Inclusion is decided at the task or subset level rather than from a dataset’s domain label.

3.2 TASK TAXONOMY

A *task configuration* specifies a task from a dataset or subset, the information available to the model, the required output, and how evidence is supplied; a dataset contributes one configuration per task format it supports. We describe each configuration along three axes.

3.2.1 HYPOTHESIS GENERATION AND SELECTION

Following Salimi et al. (2026a), **generation** produces one or more candidate explanations $H = \{h_1, \dots, h_n\}$ for observations O , whereas **selection** evaluates supplied candidates against the observations and returns the choice or subset the task requires, as in ART’s α NLI (Bhagavatula et al., 2020). Selection is posed as single choice (SCS) or, where several candidates may be correct, multiple choice (MCS). When a dataset supports both generation and selection, we evaluate each as a separate configuration. These terms describe the benchmark’s observable inputs and outputs, not stages of the model’s internal computation.

3.2.2 APPLICATION DOMAINS

Each configuration receives one primary-domain label: Healthcare, Commonsense, Narrative Reasoning, Formal Reasoning, Event Reasoning, Scientific Discovery, Scientific Reasoning, Computing Systems, Cybersecurity, or Agentic Systems (definitions in Appendix A.1).

3.2.3 EVIDENCE-DELIVERY MODE

Evidence-delivery mode describes when the model receives evidence and whether it can influence what evidence arrives next. **Static:** all evidence supplied by the task is available at the outset; it may still be incomplete, as is typical in abduction. **Passive:** evidence arrives in a predetermined sequence; the model may revise its hypothesis as evidence accumulates, but cannot choose what it receives next. **Interactive:** the model’s actions or queries determine the information it receives next, so later observations depend on the interaction history.

3.3 DATASET SELECTION AND ADAPTATION

From a larger candidate inventory, we retain tasks that meet the scope above and exclude datasets that are not publicly accessible, are prohibitively expensive to evaluate, are saturated by current models, or duplicate the abductive challenge of another selection. The resulting 27 datasets (Appendix A.2) cover all ten domains, both hypothesis formulations, and all three delivery modes. For each configuration, a dataset adapter converts source records into the common interface and connects them to the required evaluation resources, such as reference answers or an interaction environment; changes to the evidence shown, the allowed answers, or the output format are documented as our adaptations.

Passive adaptation of AthenaBench. AthenaBench poses threat-actor attribution: given evidence about malicious activity, name the responsible actor. We convert its 57 attribution cases (39 actors) into passive episodes. Each case’s 15–25 evidence items—the actor’s profile and motivation, techniques with MITRE ATT&CK identifiers, and documented incidents—are delivered in source order over four to six turns, usually four items per turn. After each turn, the model names one actor while retaining all earlier evidence and predictions; it cannot request, reorder, or stop the evidence, which separates hypothesis revision from evidence acquisition. We keep the source’s per-turn question and supply the system prompt and answer format. Because actors have many aliases (e.g., Sednit and APT28), each turn’s prediction is judged against the gold actor by an LLM judge rather than by string match (Section 4.4).

4 EVALUATION PROTOCOL

The protocol separates *final-answer evaluation*, which scores the answer or terminal result, from *process analysis*, which characterizes the reasoning trace or interaction history produced alongside it. Full metric definitions are in Appendix B.1 and all judge prompts in Appendix D.3.

4.1 PROMPTING AND TASK EXECUTION

Static tasks are run under two AbductionBench templates that replace the source prompts: input–output (IO), which requests a direct answer, and chain-of-thought (CoT), which additionally requests an explicit reasoning trace. **Passive and interactive tasks** are run as multi-turn episodes under IO only, since the interaction itself exposes the process: the source’s questions and environments are kept, while the system prompt and answer format are ours. The absence of a written trace under IO is not treated as evidence that a model performs no internal reasoning.

4.2 FINAL-ANSWER EVALUATION

Single-choice selection (SCS) is scored by accuracy, and multiple-choice selection (MCS) by exact set match (EM) and set F1 averaged over items. Generation tasks with a formally checkable answer (ProofWriter, UniADILR-HGc) are scored by exact match; other free-form explanations receive a binary correct/incorrect verdict from an LLM judge (gpt-oss-120b) that compares them with the gold explanation. Cloud-OpsBench is scored by joint root-cause accuracy: the judged fault type and the faulty component must both be correct.

4.3 REASONING-TRACE ANALYSIS

For CoT outputs, a segmentation judge splits each trace into steps, and a reasoning judge (gpt-oss-120b) scores each step or trace on thirteen descriptors: evidence use (observation coverage); step quality (useful and helpful steps, backtracking); expressed uncertainty; hypothesis breadth (branchiness, diversity, comparison exhaustiveness); direction and commitment (evidence-to-hypothesis directionality, anchoring point); unsupported content (prior knowledge, unresolved contradiction); and whether the gold hypothesis remains under consideration. These measures describe the reasoning a model writes, not its hidden computation.

4.4 PASSIVE AND INTERACTIVE EVALUATION

Every episode is scored by final-answer accuracy. For interactive tasks, each action before the final answer (a question, test, or tool call) is judged relevant or irrelevant to the gold answer, giving per-step relevance and the turns taken. For AthenaBench, every turn’s prediction is judged against the gold actor, giving per-turn accuracy, the first correct turn, and the first turn at which the final prediction appears. Static-trace measures are not applied to episodes.

Table 1: Main results by delivery protocol, task format and selection regime. Scores are percentages. **Static**: 50 records/dataset, 3 runs at temperature 0.7 (150 answers averaged); IO = direct answer, CoT = reasoning before answering. **Interactive/Passive**: 50 records/dataset, one episode, IO only (CoT = -); final-answer accuracy. Regimes: SCS = single choice, MCS = multiple choice (EM/F1), Gen = free-form generation. Metrics: ACC = accuracy, LLM-J = LLM-judge (gpt-oss-120b), EM = exact match, JRA = joint root-cause accuracy. Avg./Average = row/column means (F1 excluded from column means). Jev is a non-LLM baseline, evaluated IO-only on SCS tasks; its Average covers those six rows.

Benchmark / Dataset	Regime	Metric	GPT-5.6-Luna		Gemini-3.8-Flash		Gemma-4-31B		Qwen-3.5-27B		Gemma-4-E4B		Gemma-4-E2B		Qwen-3.5-4B		Qwen-3.5-2B		Jev	Avg.		
			IO	CoT	IO	CoT	IO	CoT	IO	CoT	IO	CoT	IO	CoT	IO	CoT	IO	CoT				
1. Static Delivery																						
1a. Selection																						
art	MCS	EM	86.7	89.3	92.7	92.7	88.7	91.3	88.0	88.7	72.7	72.0	56.7	48.0	74.0	82.7	54.0	21.3	-	75.0		
	SCS	ACC	88.0	93.3	94.0	93.3	91.8	92.2	89.3	89.6	82.4	86.7	68.7	74.7	74.4	85.8	64.2	57.8	-	82.8		
	Gen	LLM-J	43.3	50.7	49.0	44.7	51.3	42.7	44.7	52.7	32.0	39.3	26.7	37.3	32.0	41.3	14.7	8.0	-	38.2		
	ecare	MCS	EM	82.7	79.3	88.7	88.0	86.0	84.7	91.3	86.0	76.0	73.3	73.3	73.3	76.7	54.0	13.3	-	74.7		
		SCS	ACC	84.9	81.6	88.7	88.0	86.0	86.4	91.3	86.9	85.3	84.2	87.8	84.0	75.6	81.6	69.9	-	82.7		
		Gen	LLM-J	78.7	84.0	90.0	88.0	89.3	85.3	85.3	87.3	85.3	81.3	84.7	81.3	74.0	80.7	75.3	69.3	88.0	82.8	
climate_fever	MCS	EM	60.7	64.0	62.7	68.7	45.3	42.7	60.7	52.7	28.7	30.7	34.7	24.7	50.7	52.0	42.0	39.3	-	45.3		
	SCS	ACC	55.3	54.7	52.0	57.3	62.7	60.7	64.0	64.0	56.7	59.3	54.0	62.0	62.7	59.3	43.3	29.3	-	56.1		
	Gen	LLM-J	64.8	56.0	52.0	57.3	62.7	60.7	65.3	64.0	67.8	62.8	59.3	62.8	65.3	64.0	46.3	50.0	60.0	60.6		
	true_detective	SCS	ACC	40.0	51.3	55.3	56.7	62.0	60.7	67.3	61.3	70.0	64.0	62.7	58.0	64.0	67.3	39.3	64.0	62.0	62.0	
		MCS	EM	40.0	58.0	73.3	74.0	50.0	53.3	40.7	55.3	14.7	36.0	16.0	21.3	24.7	25.3	2.7	0.7	-	36.1	
		Gen	LLM-J	42.8	59.2	73.3	74.0	50.0	53.6	40.7	56.2	29.4	38.9	22.6	23.1	32.0	27.5	34.1	3.4	-	41.3	
diagnosisarena	SCS	ACC	38.7	64.0	73.3	73.3	51.3	54.0	42.0	56.1	28.7	36.0	19.3	24.7	30.0	31.3	30.0	0.7	36.0	40.0		
	MCS	EM	32.0	27.3	39.2	52.0	54.0	29.3	20.0	52.3	6.7	19.3	16.0	14.0	15.3	33.3	13.3	7.3	-	27.3		
	Gen	LLM-J	60.1	60.0	61.8	67.0	59.2	57.0	47.9	63.2	41.3	47.8	43.6	43.8	45.1	52.6	48.1	32.4	-	51.9		
	agentrx	SCS	ACC	65.3	74.7	73.3	71.3	59.3	57.3	56.0	59.3	43.3	48.7	37.3	42.0	43.3	47.3	46.0	45.3	50.7	50.7	
		MCS	EM	48.0	48.7	58.0	54.0	47.3	46.7	48.0	53.3	41.3	32.0	34.0	18.7	37.3	42.7	10.0	11.3	-	39.5	
		Gen	LLM-J	52.9	49.6	61.6	56.3	50.0	56.1	50.7	54.7	46.7	47.8	50.9	47.8	44.4	49.4	31.0	26.6	-	45.8	
aer	SCS	ACC	53.3	50.0	61.3	60.0	52.7	54.0	50.7	50.0	46.7	42.7	46.0	54.7	42.0	47.3	34.0	48.7	48.7	48.7		
	MCS	EM	56.7	64.0	65.3	66.7	73.3	66.7	49.3	62.2	49.3	58.0	38.0	43.3	52.7	58.4	37.3	3.3	-	52.5		
	Gen	LLM-J	80.3	84.2	85.1	85.7	88.4	87.3	77.6	83.8	71.6	78.7	65.6	68.2	76.3	82.0	67.4	31.5	-	75.0		
	1b. Generation																					
	NCI100	Gen	LLM-J	28.7	15.3	24.0	22.0	31.3	24.7	32.7	24.7	26.0	26.7	28.0	28.7	22.7	24.0	14.7	20.0	-	24.6	
		Gen	LLM-J	62.0	70.7	83.3	81.3	50.0	54.7	59.3	53.3	38.7	46.0	28.0	28.7	35.3	32.7	23.3	18.7	-	47.9	
Gen		LLM-J	62.0	66.7	56.3	56.7	61.3	53.3	53.3	34.0	38.7	32.0	44.0	32.7	42.0	42.7	19.3	13.3	-	44.3		
house_md	Gen	LLM-J	86.0	95.3	90.5	88.0	94.0	90.0	83.3	83.8	59.3	73.3	59.3	80.0	65.3	86.7	52.7	74.7	-	78.9		
	Gen	LLM-J	38.0	42.7	43.3	54.7	40.7	33.3	26.0	40.7	15.3	19.3	10.0	14.0	11.3	21.3	5.3	4.0	-	26.2		
	Gen	LLM-J	78.7	71.3	72.7	74.7	71.3	71.3	68.0	54.1	58.7	54.0	50.0	51.3	53.3	54.0	38.7	29.3	-	59.5		
llm4biohypogen	Gen	LLM-J	70.0	64.0	67.1	66.0	62.0	62.7	71.3	68.2	63.3	65.3	56.7	60.0	62.0	67.3	36.7	52.7	-	62.2		
	Gen	LLM-J	76.7	64.7	59.3	60.7	69.3	53.3	57.3	57.9	47.3	40.7	41.3	45.3	48.7	42.7	23.3	12.0	-	57.0		
	Gen	LLM-J	62.0	66.0	54.4	60.7	49.3	54.0	42.0	37.5	25.3	14.7	18.7	23.3	42.7	8.0	20.7	-	37.5			
matter_to_mechanism	Gen	LLM-J	87.3	90.0	88.0	82.7	69.3	79.3	78.0	77.3	54.0	60.7	56.7	64.0	56.0	67.3	29.3	39.3	-	60.9		
	Gen	LLM-J	58.7	64.7	68.7	68.0	54.0	70.7	52.7	66.7	39.0	64.0	28.0	62.0	13.3	68.0	3.3	37.3	-	50.7		
	Gen	EM	37.0	40.0	40.0	44.0	42.0	47.3	34.0	59.3	39.3	17.3	15.3	52.7	16.0	52.0	7.3	8.7	-	60.5		
uncommonsense	Gen	LLM-J	78.7	78.7	71.3	70.0	66.0	58.0	74.7	63.4	73.3	68.7	64.0	55.3	48.7	47.3	34.7	18.7	-	50.7		
	Gen	EM	42.7	33.3	72.0	76.0	64.0	46.7	14.0	14.3	40.0	2.7	9.3	4.0	13.3	8.0	0.7	3.3	-	27.8		
2. Interactive Delivery																						
medqdx	Gen	LLM-J	40.0	-	34.0	-	36.0	-	32.0	-	24.0	-	32.0	-	26.0	-	6.0	-	-	-	28.8	
	Gen	LLM-J	22.0	-	44.0	-	28.0	-	29.0	-	12.0	-	8.0	-	6.0	-	0.0	-	-	-	17.5	
	Gen	LLM-J	52.0	-	58.0	-	48.0	-	40.0	-	22.0	-	18.0	-	28.0	-	8.0	-	-	-	37.5	
	viva_bench	SCS	ACC	32.0	-	22.0	-	26.0	-	42.0	-	36.0	-	44.0	-	40.0	-	18.0	-	-	-	32.5
		MCS	ACC	28.0	-	30.0	-	36.0	-	38.0	-	34.0	-	38.0	-	32.0	-	24.0	-	-	-	32.5
		Gen	JRA	6.0	-	10.0	-	4.0	-	2.0	-	0.0	-	0.0	-	0.0	-	0.0	-	-	-	2.8
3. Passive Delivery																						
athena_bench	Gen	LLM-J	30.0	-	44.0	-	26.0	-	20.0	-	20.0	-	8.0	-	10.0	-	0.0	-	-	-	19.8	
	Average																					
			54.8	62.6	60.0	67.0	55.4	59.3	51.1	58.9	41.4	47.8	36.6	44.1	38.9	51.4	25.4	27.2	62.0	-	-	

5 RESULTS

We evaluate 27 datasets—21 static, five interactive, and one passive—on 50 seeded records each (Appendix A). The eight LLMs are GPT-5.6-Luna, Gemini-3.8-Flash, Gemma-4-31B, and Qwen-3.5-27B, queried through an API, and Gemma-4-E4B/E2B and Qwen-3.5-4B/2B, served locally with vLLM; the Jev decision system answers the single-choice tasks under IO only. Static tasks are run under IO and CoT, with three samples per record at temperature 0.7. Interactive and passive tasks are run once per record under IO, with native reasoning disabled (minimal for Gemini-3.8-Flash, which cannot disable it) and the full interaction history in every turn, up to 14 turns for MedQDx, six for AthenaBench, and 20 otherwise; gpt-4o-mini plays the patient or examiner in MedQDx, Med-Inquire, and VivaBench, DDX-Plus answers from each patient’s recorded questionnaire by BM25 retrieval, and CloudOps-Bench replays its recorded tool outputs. Free-form answers and all trace and step-relevance measures are judged by gpt-oss-120b (Appendix D). Empty or truncated replies count as incorrect. For Qwen-3.5-27B under CoT, provider errors left only 7–101 of 150 answers on ten generation datasets (house_md, hypogen, llm4biohypogen, matter_to_mechanism, medcasereasoning, medr_bench, neulr, proof_writer, uncommonsense, uniadilr_hgc); these cells average the completed answers only, are never bolded, and should be read with caution.

Table 1 reports final-answer scores for eight LLMs and the Jev decision system; the subsections below explain what drives them and what the reasoning traces and interaction histories add.

5.1 WHAT DETERMINES PERFORMANCE ACROSS BENCHMARKS?

Task format. Proposing an explanation is harder than recognizing it: where one dataset is posed both ways, the row average falls from 84.5 (SCS) to 38.2 (Gen) on art and from

82.8 to 47.5 on **ecare**. Allowing several answers also costs accuracy for every model—10.6 points on average, up to 27.7 for Qwen-3.5-2B (Appendix C.1.1).

Chain-of-thought. Averaged over the static rows, CoT raises accuracy for six of the eight LLMs—most for Qwen-3.5-4B (43.4→51.4)—and lowers it for Gemma-4-31B and Qwen-3.5-2B (2.4 points each). It helps where the explanation must be derived by chaining premises or findings—**neulr** (Qwen-3.5-4B 13.3→68.0), **proof_writer** (Gemma-4-E2B 15.3→52.7), **medcasereasoning** (Qwen-3.5-4B 23.3→42.7)—but often hurts open-ended hypothesis generation: **uniadilr_hgc** (Gemma-4-E4B 40.0→2.7), **matter_to_mechanism** (GPT-5.6-Luna 76.7→64.7), **uncommonsense** (Qwen-3.5-2B 34.7→18.7).

Delivery mode and models. Interactive and passive tasks are among the hardest: the best model reaches 58.0 on **vivabench**, 44.0 on **athena_bench**, and only 10.0 on **cloud_opsbench**, where four of the eight LLMs score 0.0. Gemini-3.8-Flash has the highest average in both modes and the best score on four of the seven interactive and passive rows; the smallest model, Qwen-3.5-2B, is last. IO averages include the interactive and passive rows, which have no CoT variant, so the IO and CoT columns of the Average row are not directly comparable.

5.2 CAN A SYSTEM ONE DECISION SYSTEM PERFORM ABDUCTIVE SELECTION?

Only in a narrow sense. Jev chooses among the supplied candidates without writing an explanation, so we evaluate it only on SCS, the discriminative half of abduction. On the six single-choice tasks it averages 62.0, above every small LLM under IO (48.3–59.3) but below every large one (64.7–74.5). It is near the top on short commonsense explanation choices—88.0 on **ecare**, where only two of the sixteen LLM cells are higher, and 86.7 on **art**—but falls far behind when the candidates must be weighed against many clues in a long case: 36.0 vs. 73.3 on **true_detective** and 50.7 vs. 74.7 on **diagnosisarena**. A fast decision system thus covers explanation *selection* over compact evidence, but not the evidence integration that longer abductive cases require.

5.3 WHICH TRACE PROPERTIES ACCOMPANY A CORRECT EXPLANATION?

For Gemini-3.8-Flash, Gemma-4-31B, and GPT-5.6-Luna, correct and incorrect CoT traces differ most in whether the correct explanation stays in play: the helpful-step fraction (0.72 vs. 0.19) and the rate at which the gold hypothesis remains under consideration (0.36 vs. 0.16) drop sharply in failures. Both are judged against the gold answer and partly restate correctness, but gold-free measures point the same way: correct traces cover more observations (0.72 vs. 0.61) and compare candidates more exhaustively (0.62 vs. 0.52), whereas the *form* of the trace—directionality, prior-knowledge use, anchoring point—barely differs (Appendix C.3). Longer traces do not help either: within each dataset, accuracy falls from 68% for the shortest fifth of CoT replies to 37% for the longest, for every one of the eight LLMs, and the decline beyond about six steps holds at every input length (Appendix C.3.1). The relation is associational—harder cases elicit longer traces—but additional steps do not recover an explanation the model has not found early.

5.4 WHERE IN THE CHAIN DO CORRECT AND INCORRECT REASONING DIVERGE?

On average, traces follow the order abduction prescribes (Figure 1a): observations concentrate in the first fifth of the chain, backtracking and uncertainty peak near the midpoint, and the anchoring point, the gold hypothesis, and step directionality peak in the last fifth—evidence first, commitment last. Splitting by outcome (Figure 1b) shows where failures begin. *Unresolved contradiction* is higher in incorrect chains from the very first steps, before any hypothesis is anchored, making it the earliest trace signal of an eventual error. Around the midpoint, uncertainty, backtracking, branchiness, and observation mentions are all higher in incorrect chains, so mid-chain hedging and revisiting accompany wrong answers rather than successful self-correction. The gold hypothesis and helpful steps favor correct chains throughout and increasingly toward the end, partly by construction.

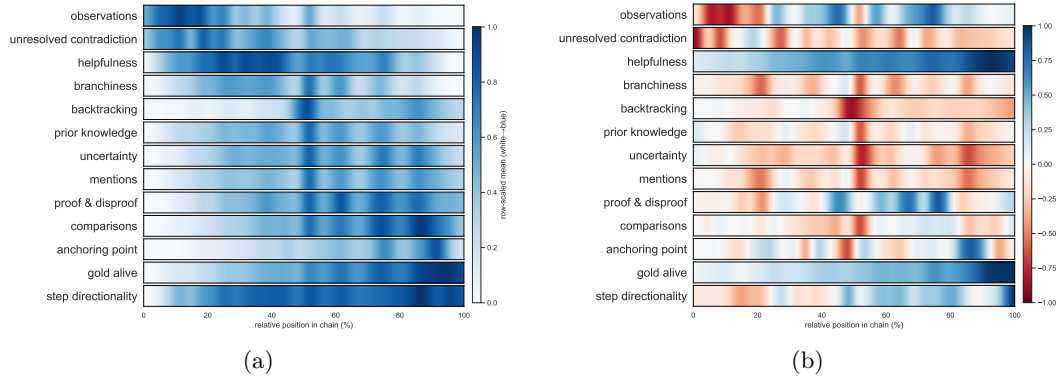


Figure 1: Judged trace measures by relative position in the chain (0–100%), pooled over the eight LLMs. **(a)** Mean, each row min-max scaled. **(b)** Correct-minus-incorrect difference, computed within each dataset and averaged; each row scaled symmetrically around zero, so **blue** marks positions where correct chains score higher and **red** where incorrect chains score higher. Row intensities are not comparable across measures.

5.5 DOES ASKING THE RIGHT QUESTION FIRST DECIDE AN INTERACTIVE DIAGNOSIS?

Largely, yes. Only 35% of interactive episodes open with an action judged relevant to the gold answer, but those end correctly 2.4 times as often as the rest (39.7% vs. 16.6%), and every model shows the effect, with ratios from $\times 1.8$ (Gemma-4-31B, Qwen-3.5-27B) to $\times 3.1$ (Qwen-3.5-2B) (Figure 2). Relevance then declines as the episode proceeds for every model (Appendix C.2), so later questions rarely repair a poor start. The relation is associational—a model that understands the case both asks better and diagnoses better—but it shows that the quality of evidence *seeking*, invisible in final-answer accuracy, is measurable from the first step.

5.6 DO MODELS REVISE THEIR HYPOTHESIS AS PASSIVE EVIDENCE ACCUMULATES?

Rarely. Each AthenaBench case reveals 15–25 evidence items over four to six turns, yet in 78.5% of the 400 episodes the model names the same actor at every turn (Figure 3). Revisions seldom help: only 7 episodes move from a wrong first prediction to a correct final one, and 2 move the other way, so final accuracy nearly equals first-turn accuracy (Gemini-3.8-Flash 40%→44%, GPT-5.6-Luna 24%→30%, unchanged for the other six LLMs). Frequent revision is not sufficient either: Qwen-3.5-2B changes its prediction in 48% of episodes but never reaches the correct actor. Passive delivery thus exposes what static evaluation cannot—models anchor on the explanation formed from the first evidence and do not update it as the evidence accumulates.

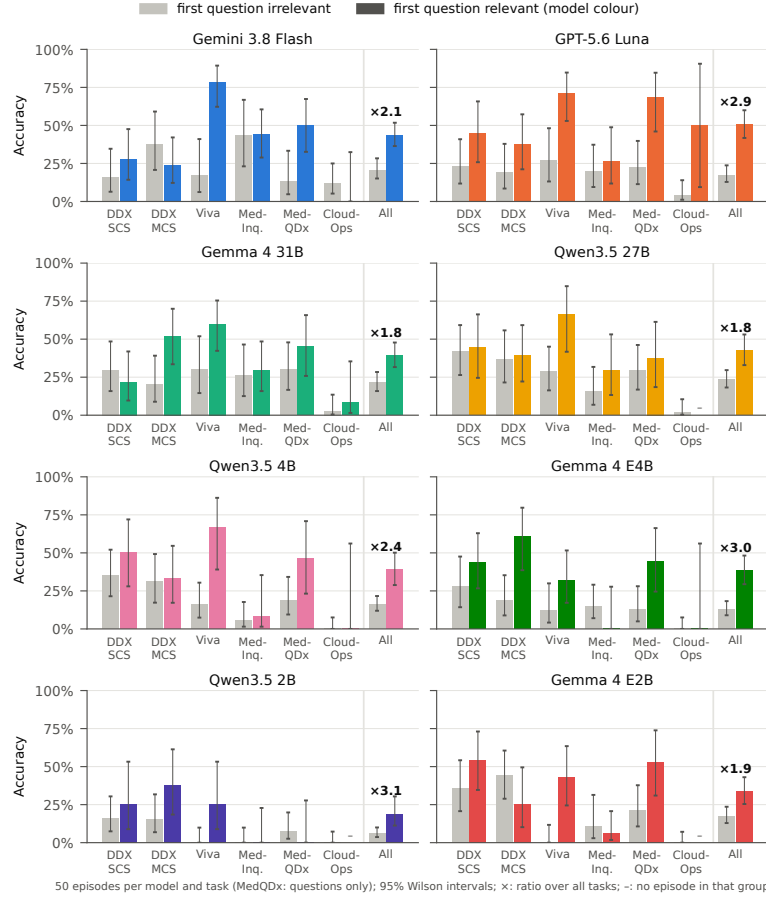


Figure 2: Final-answer accuracy of interactive episodes whose *first* action was judged irrelevant (gray) or relevant (color) to the gold answer, per task and pooled (*All*), per model; 95% Wilson intervals. ×: relevant-to-irrelevant ratio over all tasks.

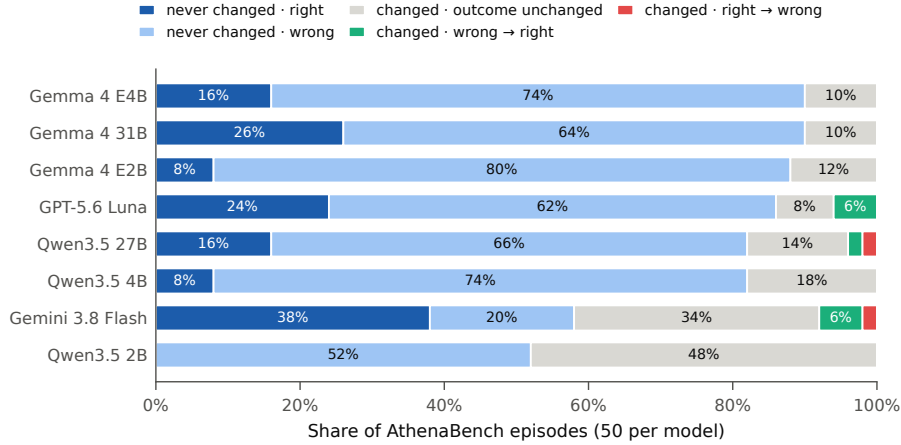


Figure 3: AthenaBench (passive): share of the 50 episodes per model whose predicted actor never changed (right or wrong from the first turn), changed without changing the outcome, or changed from wrong to right or right to wrong between the first and last turn.

6 LIMITATIONS

Comparisons across domains, hypothesis formulations, and delivery modes are observational: the source datasets differ in content, difficulty, and answer space, so category-level differences do not isolate a single cause. All free-form answers, trace measures, and step-relevance labels come from a single LLM judge (gpt-oss-120b) whose agreement with human annotators we have not measured. With 50 records per dataset, a static score has a 95% interval of roughly ± 7 –8 points and an episode score of up to ± 13 , so small differences between models should not be over-read. Interactive results depend on the simulated patient or examiner (gpt-4o-mini) and on recorded environments, and passive and interactive tasks are run under IO only. Finally, written traces are observable outputs, not faithful records of the computation that produced the answer. Future work should validate the judges on human-annotated samples and add controlled variants that change one task property at a time.

7 CONCLUSION

AbductionBench organizes abductive evaluation around what models must do with hypotheses, where tasks are applied, and how evidence becomes available, and it reports reasoning traces and interaction histories alongside final answers. Across eight LLMs and 27 datasets, generating an explanation is much harder than selecting one, CoT helps derivation-heavy tasks but often hurts open-ended generation, and longer traces go with lower accuracy. A non-LLM decision system matches small LLMs on selection over compact evidence but not on long cases. Beyond static tasks, the first question of an interactive diagnosis already signals its outcome, and models rarely revise their first hypothesis when evidence arrives passively. Evaluating abduction therefore requires looking at how evidence is sought and used, not only at the final answer, while keeping clear limits on what written traces reveal about internal reasoning.

AI USE STATEMENT

In this work, LLMs are the object of study: the evaluated models, the answer and reasoning-trace judge (gpt-oss-120b), and the simulated patients and examiners (gpt-4o-mini) are described in Section 5 and Appendix D. Beyond this, we used generative AI coding assistants to help implement the evaluation framework, dataset adapters, and analysis and plotting scripts, and to assist with editing the text of this paper. We did not use generative AI tools to generate research ideas or to fabricate or alter data or results. All AI-assisted code was reviewed and tested by the authors, all reported numbers are computed by our pipeline from the stored model outputs, and all AI-assisted text was reviewed and revised by the authors. We take responsibility for the final content of this work, including text, claims, and artifacts produced with the aid of generative AI.

ETHICS STATEMENT

AbductionBench is built from publicly released datasets, used under their licenses; we add no new human-subject data and collect no personal information. The clinical datasets contain de-identified or synthetic cases, and model outputs on them are not medical advice. AthenaBench describes real threat actors and their publicly reported techniques; we use it only to evaluate attribution and release no new offensive capability. Simulated patients and examiners are played by an LLM, not by people. Automated judging can carry the judge model’s biases, which we note as a limitation.

REPRODUCIBILITY STATEMENT

Section 3 and Appendix A.2 list every dataset, configuration, and adaptation; Section 5 and Appendix D give the models, decoding settings, turn limits, simulators, and judges; Appendix B.1 defines every metric; and Appendix D.3 reproduces the judge prompts. Records are sampled with a fixed seed, and every response, judge verdict, and metric is stored per sample. We will release the evaluation framework, dataset adapters, sampled record

identifiers, and all model outputs upon publication; an anonymized copy is provided as supplementary material.

REFERENCES

- Md Tanvirul Alam, Dipkamal Bhusal, Salman Ahmad, Nidhi Rastogi, and Peter Worth. Athenabench: A dynamic benchmark for evaluating llms in cyber threat intelligence, 2026. URL <https://arxiv.org/abs/2511.01144>.
- Ahmed Almasoud. Evaluating multi-agent llm architectures for rare disease diagnosis, 2026. URL <https://arxiv.org/abs/2603.06856>.
- Jiaxin Bai, Yicheng Wang, Tianshi Zheng, Yue Guo, Xin Liu, and Yangqiu Song. Advancing abductive reasoning in knowledge graphs through complex logical hypothesis generation. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1312–1329, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.72. URL <https://aclanthology.org/2024.acl-long.72/>.
- Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. Agentrx: Diagnosing ai agent failures from execution trajectories, 2026. URL <https://arxiv.org/abs/2602.02475>.
- Serafim Batzoglou. Abd: Default exception abduction in finite first order worlds, 2026. URL <https://arxiv.org/abs/2602.18843>.
- Chandra Bhagavatula, Ronan Le Bras, Chaitanya Malaviya, Keisuke Sakaguchi, Ari Holtzman, Hannah Rashkin, Doug Downey, Scott Wen-tau Yih, and Yejin Choi. Abductive commonsense reasoning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=Byg1v1HKDB>.
- Andrew Bouras and OMS-II Research Fellow. Crosstrace: A cross-domain dataset of grounded scientific reasoning traces for hypothesis generation, 2026. URL <https://arxiv.org/abs/2603.28924>.
- Yuanhong Cai, Xiaohui Nie, Kanglin Yin, Changhua Pei, Yongqian Sun, Shenglin Zhang, Haibin Liu, Guiyang Liu, Xidao Wen, Fang Situ, and Dan Pei. A multi-dataset benchmark for evaluating llm agents in microservice failure diagnosis, 2026. URL <https://arxiv.org/abs/2606.29193>.
- Pengfei Cao, Mingxuan Yang, Yubo Chen, Chenlong Zhang, Mingxuan Liu, Kang Liu, and Jun Zhao. SemEval-2026 task 12: Abductive event reasoning: Towards real-world event causal inference for large language models. In Ekaterina Kochmar, Debanjan Ghosh, Kai North, Mamoru Komachi, and Marcos Zampieri (eds.), *Proceedings of the 20th International Workshop on Semantic Evaluation (2026)*, pp. 3659–3672, San Diego, California, USA, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-414-9. doi: 10.18653/v1/2026.semeval-1.446. URL <https://aclanthology.org/2026.semeval-1.446/>.
- Tingting Chen, Beibei Lin, Zifeng Yuan, Qiran Zou, Hongyu He, Anirudh Goyal, Yew-Soon Ong, and Dianbo Liu. Hypospace: A diagnostic benchmark for set-valued hypothesis generation under underdetermination and sublinear coverage bounds. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=QpjtK65JH0>.
- Aditya Chinchure, Sahithya Ravi, Raymond Ng, Vered Shwartz, Boyang Li, and Leonid Sigal. Black swan: Abductive and defeasible video reasoning in unpredictable events. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 24201–24210, June 2025.

- Christopher Chiu, Silviu Pitis, and Mihaela van der Schaar. Simulating viva voce examinations to evaluate clinical reasoning in large language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2026. URL <https://openreview.net/forum?id=FEVfIPMy5b>.
- Patrick Cooper and Alvaro Velasquez. Defab: A verifiable benchmark for defeasible abduction in foundation models, 2026. URL <https://arxiv.org/abs/2606.18557>.
- Dhairya Dalal, Marco Valentino, Andre Freitas, and Paul Buitelaar. Inference to the best explanation in large language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 217–235, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.14. URL <https://aclanthology.org/2024.acl-long.14/>.
- Maksym Del and Mark Fishel. True detective: A deep abductive reasoning benchmark undoable for GPT-3 and challenging for GPT-4. In Alexis Palmer and Jose Camacho-collados (eds.), *Proceedings of the 12th Joint Conference on Lexical and Computational Semantics (*SEM 2023)*, pp. 314–322, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.starsem-1.28. URL <https://aclanthology.org/2023.starsem-1.28>.
- Thomas Diggelmann, Jordan Boyd-Graber, Jannis Bulian, Massimiliano Ciaramita, and Markus Leippold. Climate-fever: A dataset for verification of real-world climate claims, 2021. URL <https://arxiv.org/abs/2012.00614>.
- John Dougrez-Lewis, Mahmud Elahi Akhter, Federico Ruggeri, Sebastian Löbbers, Yulan He, and Maria Liakata. Assessing the reasoning capabilities of LLMs in the context of evidence-based claim verification. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 20604–20628, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi: 10.18653/v1/2025.findings-acl.1059. URL <https://aclanthology.org/2025.findings-acl.1059/>.
- Li Du, Xiao Ding, Kai Xiong, Ting Liu, and Bing Qin. e-CARE: a new dataset for exploring explainable causal reasoning. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (eds.), *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 432–446, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.33. URL <https://aclanthology.org/2022.acl-long.33/>.
- Kanishk Gandhi, Michael Y. Li, Lyle Goodyear, Agam Bhatia, Ying Li, Aditi Bhaskar, Mohammed Zaman, and Noah Goodman. Boxinggym: Benchmarking progress in automated experimental design and model discovery. In *Workshop on Scaling Environments for Agents*, 2025. URL <https://openreview.net/forum?id=TgobzsU03X>.
- Taolin Han, Shuang Wu, Jinghang Wang, Yuhao Zhou, Renquan Lv, Bing Zhao, and Wei Hu. Molquest: A benchmark for agentic evaluation of abductive reasoning in chemical structure elucidation, 2026. URL <https://arxiv.org/abs/2603.25253>.
- Kaiyu He, Peilin Wu, Mian Zhang, Kun Wan, Wentian Zhao, Xinya Du, and Zhiyu Chen. Gear: A general evaluation framework for abductive reasoning, 2025. URL <https://arxiv.org/abs/2509.24096>.
- Yufei He, Juncheng Liu, Zhiyuan Hu, Yulin Chen, Yue Liu, Yuan Sui, Yibo Li, Nuo Chen, Jun Hu, Bryan Hooi, Xinxing Xu, and Jiang Bian. Evoclinician: A self-evolving agent for multi-turn medical diagnosis via test-time evolutionary learning, 2026. URL <https://arxiv.org/abs/2601.22964>.
- Haokun Liu, Sicong Huang, Jingyu Hu, Yangqiaoyu Zhou, and Chenhao Tan. Hypobench: Towards systematic and principled benchmarking for hypothesis generation, 2026. URL <https://arxiv.org/abs/2504.11524>.

- Charles O'Neill, Tirthankar Ghosal, Roberta Răileanu, Mike Walmsley, Thang Bui, Kevin Schawinski, and Ioana Ciucă. Sparks of science: Hypothesis generation using structured paper data, 2025. URL <https://arxiv.org/abs/2504.12976>.
- Biqing Qi, Kaiyan Zhang, Kai Tian, Haoxiang Li, Zhang-Ren Chen, Sihang Zeng, Ermo Hua, Hu Jinfang, and Bowen Zhou. Large language models as biomedical hypothesis generators: A comprehensive evaluation. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=q36rpG1G9X>.
- Pengcheng Qiu, Chaoyi Wu, Shuyu Liu, Yanjie Fan, Weike Zhao, Zhuoxia Chen, Hongfei Gu, Chuanjin Peng, Ya Zhang, Yanfeng Wang, and Weidi Xie. Quantifying the reasoning abilities of llms on clinical cases. *Nature Communications*, 16(1):9799, Nov 2025. ISSN 2041-1723. doi: 10.1038/s41467-025-64769-1. URL <https://doi.org/10.1038/s41467-025-64769-1>.
- Moein Salimi, Shaygan Adim, Danial Parnian, Nima Alighardashi, Mahdi Jafari Siavoshani, and Mohammad Hossein Rohban. Wiring the ‘why’: A unified taxonomy and survey of abductive reasoning in LLMs. *Transactions on Machine Learning Research*, 2026a. ISSN 2835-8856. URL <https://openreview.net/forum?id=oeVkugHOWB>. Survey Certification.
- Moein Salimi, Danial Parnian, Shaygan Adim, Amirmohammad Ebrahimirasab, Nima Alighardashi, Parsa Gholami, Sahand Akramipour, Mahdi Jafari Siavoshani, and Mohammad Hossein Rohban. Cedar-grpo: Process-aware reinforcement learning for general abductive reasoning in llms, 2026b. URL <https://arxiv.org/abs/2608.14791>.
- G. Schurz. Patterns of abduction. *Synthese*, 164(2):201–234, Sep 2008. ISSN 1573-0964. doi: 10.1007/s11229-007-9223-4. URL <https://doi.org/10.1007/s11229-007-9223-4>.
- Yu Sheng, Wanting Wen, Linjing Li, and Daniel Zeng. Evaluating generalization capability of language models across abductive, deductive and inductive logical reasoning. In Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (eds.), *Proceedings of the 31st International Conference on Computational Linguistics*, pp. 4945–4957, Abu Dhabi, UAE, January 2025. Association for Computational Linguistics. URL <https://aclanthology.org/2025.coling-main.330/>.
- Shashwat Sourav, Tanjin. He, Maria K. Y. Chan, Anubhav Jain, and Tirthankar Ghosal. Matter to mechanism: A benchmark for ai co-scientists in materials and battery research, 2026. URL <https://arxiv.org/abs/2606.02258>.
- Zayne Sprague, Kaj Bostrom, Swarat Chaudhuri, and Greg Durrett. Natural language deduction with incomplete information. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (eds.), *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 8230–8258, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.564. URL <https://aclanthology.org/2022.emnlp-main.564/>.
- Zayne Rea Sprague, Xi Ye, Kaj Bostrom, Swarat Chaudhuri, and Greg Durrett. MuSR: Testing the limits of chain-of-thought with multistep soft reasoning. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=jenyYQzue1>.
- Donald E Stanley and Rune Nyrupe. Strategies in abduction: Generating and selecting diagnostic hypotheses. *The Journal of Medicine and Philosophy: A Forum for Bioethics and Philosophy of Medicine*, 45(2):159–178, 04 2020. ISSN 0360-5310. doi: 10.1093/jmp/jhz041. URL <https://doi.org/10.1093/jmp/jhz041>.
- Julius Steiglechner, Lucas Mahler, and Gabriele Lohmann. Llms can see the smoke but not the fire: Evaluating abductive reasoning with elenchos, 2026. URL <https://arxiv.org/abs/2607.12733>.
- Oyvind Tafjord, Bhavana Dalvi, and Peter Clark. ProofWriter: Generating implications, proofs, and abductive statements over natural language. In Chengqing Zong,

- Fei Xia, Wenjie Li, and Roberto Navigli (eds.), *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pp. 3621–3634, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.findings-acl.317. URL <https://aclanthology.org/2021.findings-acl.317/>.
- Arsène Fansi Tchango, Rishab Goel, Zhi Wen, Julien Martel, and Joumana Ghosn. Ddxplus: a new dataset for automatic medical diagnosis. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS ’22, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.
- Armin Toroghi, Faeze Moradi Kalarde, and Scott Sanner. Commonwhy: A dataset for evaluating entity-based causal commonsense reasoning in large language models. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR ’26, pp. 3418–3425, New York, NY, USA, 2026. Association for Computing Machinery. ISBN 9798400725999. doi: 10.1145/3805712.3808628. URL <https://doi.org/10.1145/3805712.3808628>.
- Yilun Wang, Guangba Yu, Haiyu Huang, Yujie Huang, Zirui Wang, Pengfei Chen, and Michael R. Lyu. Cloud-opsbench: A reproducible benchmark for agentic root cause analysis in cloud systems, 2026. URL <https://arxiv.org/abs/2603.00468>.
- Mai Werthaim, Maya Kimhi, Alexander Apartsin, and Yehudit Aperstein. A benchmark for evaluating diagnostic questioning efficiency of llms in patient conversations. *Scientific Reports*, 16(1):6121, Jan 2026. ISSN 2045-2322. doi: 10.1038/s41598-026-37022-y. URL <https://doi.org/10.1038/s41598-026-37022-y>.
- Kevin Wu, Eric Wu, Rahul Thapa, Kevin Wei, Angela Zhang, Arvind Suresh, Jacqueline J. Tao, Min Woo Sun, Alejandro Lozano, and James Zou. Medcasereasoning: Evaluating and learning diagnostic reasoning from clinical case reports, 2025. URL <https://arxiv.org/abs/2505.11733>.
- Fangzhi Xu, Qika Lin, Jiawei Han, Tianzhe Zhao, Jun Liu, and Erik Cambria. Are large language models really good logical reasoners? a comprehensive evaluation and beyond. *IEEE Trans. Knowl. Data Eng.*, 37(4):1620–1634, April 2025. URL <https://doi.org/10.1109/TKDE.2025.3536008>.
- Nathan Young, Qiming Bao, Joshua Bensemann, and Michael Witbrock. Abduction-Rules: Training transformers to explain unexpected inputs. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (eds.), *Findings of the Association for Computational Linguistics: ACL 2022*, pp. 218–227, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.findings-acl.19. URL <https://aclanthology.org/2022.findings-acl.19/>.
- Wenting Zhao, Justin T. Chiu, Jena D. Hwang, Faeze Brahman, Jack Hessel, Sanjiban Choudhury, Yejin Choi, Xiang Lorraine Li, and Alane Suhr. UNcommonsense reasoning: Abductive reasoning about uncommon situations. In Kevin Duh, Helena Gomez, and Steven Bethard (eds.), *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 8487–8505, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.469. URL <https://aclanthology.org/2024.naacl-long.469/>.
- Yakun Zhu, Zhongzhen Huang, Linjie Mu, Yutong Huang, Wei Nie, Jiaji Liu, Shaoting Zhang, Pengfei Liu, and Xiaofan Zhang. DiagnosisArena: Benchmarking diagnostic reasoning for large language models. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 3074–3098, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-395-1. doi: 10.18653/v1/2026.findings-acl.151. URL <https://aclanthology.org/2026.findings-acl.151/>.

A BENCHMARK INVENTORY AND TAXONOMY

A.1 TAXONOMY AND DOMAIN DEFINITIONS

These definitions preserve the working domain taxonomy. Each evaluated task receives one primary label; Because some tasks naturally fit into multiple domains, our mapping rules explicitly resolve these overlaps to ensure each task receives exactly one primary label. Furthermore, fitting into a domain category is not sufficient for inclusion; every selected dataset must still strictly require abductive reasoning, as defined in Section 3.1.

Our collection spans several application domains, each requiring the model to infer a different kind of hidden cause or explanation. We define each domain below.

Healthcare comprises tasks in which a model infers a diagnosis, differential, or next clinical action from patient symptoms, history, test results, or an ongoing clinical conversation.

Commonsense comprises tasks in which a model selects or generates the more plausible everyday cause or explanation for a short observed situation, without requiring specialized domain knowledge.

Narrative Reasoning comprises tasks in which a model infers an explanation that accounts for clues or events scattered across a longer story or mystery.

Formal Reasoning comprises tasks in which a model generates or selects a missing fact, rule, premise, or hypothesis within a symbolic or logical system, checkable against a formally defined ground truth.

Event Reasoning comprises tasks in which a model infers the plausible cause or causes of an observed real-world event from a description of that event.

Scientific Discovery comprises tasks in which a model proposes a hypothesis or mechanism that could explain empirical or literature-grounded observations, typically without an established ground-truth answer.

Scientific Reasoning comprises tasks in which a model evaluates or infers the degree of support that a body of evidence lends to a scientific claim or hypothesis, spanning multiple reasoning paradigms.

Computing Systems comprises tasks in which a model infers a plausible underlying cause of a system fault, performance anomaly, or program behavior from observations such as logs, stack traces, telemetry, or binary.

Cybersecurity comprises tasks in which a model infers a plausible attack, intrusion cause, or responsible actor from incomplete or noisy security-event evidence.

Agentic Systems comprises tasks in which a model infers which step or steps of an AI agent’s trajectory account for its eventual outcome, whether by assigning credit for success or diagnosing the cause of a failure.

Planning comprises tasks in which a model infers unstated actions or temporal dependencies that would lead to a stated goal.

A.2 DATASET AND TASK INVENTORY

Table 2: Summary of datasets used. Data Delivery Mode: **S** = Static, **P** = Passive, **I** = Interactive. Hypothesis Processing Mode: **G** = Generation, **Sel** = Selection; **G / Sel** denotes datasets with separate generation and selection tasks.

#	Name and Citation	Domain	Data Delivery	Hyp. Mode
1	MedQDx Werthaim et al. (2026)	Healthcare: Diagnostic Questioning	I	G

Continued on next page

#	Name and Citation	Domain	Data Delivery	Hyp. Mode
2	Med-Inquire He et al. (2026)	Healthcare: Clinical Diagnosis	I	G
3	Cloud-OpsBench Wang et al. (2026)	Computing Systems: Cloud Root-Cause Analysis	I	G
4	VivaBench Chiu et al. (2026)	Healthcare: Clinical Diagnosis	I	G
5	e-CARE Du et al. (2022)	Commonsense: Causal Reasoning	S	G / Sel
6	ART Bhagavatula et al. (2020)	Narrative Reasoning: Abductive Explanation	S	G / Sel
7	UNCOMMONSENSE Zhao et al. (2024)	Commonsense: Uncommon-Situation Abduction	S	G
8	ProofWriter Tafjord et al. (2021)	Formal Reasoning: Rule-Based Inference	S	G
9	True Detective Del & Fishel (2023)	Narrative Reasoning: Mystery Solving	S	Sel
10	ENWN and EntailmentBank Sprague et al. (2022)	Formal Reasoning: Missing-Premise Inference	S	G
11	DiagnosisArena Zhu et al. (2026)	Healthcare: Clinical Diagnosis	S	Sel
12	MedR-Bench Qiu et al. (2025)	Healthcare: Clinical Reasoning	S	G
13	NeuLR Xu et al. (2025)	Formal Reasoning: Logical Entailment	S	G
14	DDXPlus Tchango et al. (2022)	Healthcare: Differential Diagnosis	I	Sel
15	AER Cao et al. (2026)	Event Reasoning: Abductive Event Causality	S	Sel
16	House M.D. Almasoud (2026)	Healthcare: Rare-Disease Diagnosis	S	G
17	CrossTrace Bouras & Fellow (2026)	Scientific Discovery: Cross-Domain Hypothesis Generation	S	G
18	HypoGen O'Neill et al. (2025)	Scientific Discovery: Computer-Science Hypothesis Generation	S	G
19	Matter to Mechanism Sourav et al. (2026)	Scientific Discovery: Materials Mechanism Inference	S	G
20	AgentRx Barke et al. (2026)	Agentic Systems: Agent Failure Diagnosis	S	Sel
21	RCA100 Cai et al. (2026)	Computing Systems: Microservice Failure Diagnosis	S	G
22	UniADILR-HGc Sheng et al. (2025)	Formal Reasoning: Constrained Abductive Hypothesis Generation	S	G
23	CLIMATE-FEVER Diggelmann et al. (2021)	Scientific Reasoning: Evidence-Based Claim Explanation	S	Sel
24	AthenaBench (TAA) Alam et al. (2026)	Cybersecurity: Threat Actor Attribution	P	G
25	MedCaseReasoning Wu et al. (2025)	Healthcare: Clinical Diagnosis	S	G
26	CommonWhy Toroghi et al. (2026)	Commonsense: Entity-Based Causal Reasoning	S	G

Continued on next page

#	Name and Citation	Domain	Data Delivery	Hyp. Mode
27	LLM4BioHypoGen Qi et al. (2024)	Scientific Discovery: Biomedical Hypothesis Generation	S	G

A.3 BENCHMARK COVERAGE AND RELATED EVALUATION EFFORTS

A.3.1 OPERATIONAL CAPABILITY TAXONOMY

We distinguish four groups of evaluation targets: *formation* (constructing hypotheses and covering alternatives); *assessment* (evidence support, consistency, contextual plausibility, and explicitly defined parsimony); *discrimination* (selection, ranking, verification, and calibrated uncertainty); and *revision and use* (theory repair, updating under new evidence, predictive testing, and evidence acquisition). This is an operational synthesis, not a claim that these are independent latent abilities or a replacement for the generation–selection distinction (Schurz, 2008; Stanley & Nyrup, 2020; Salimi et al., 2026a).

Task descriptors must be kept separate from measured capabilities. These include output representation, supplied versus constructed candidate spaces, static/passive/interactive delivery, domain, and modality. In particular, free-form output need not imply an unrestricted hypothesis space; formal open-world semantics concerns unknown facts, not whether answer options are supplied. Multi-step inference and evidence integration should be credited as independently measured capabilities only when the protocol varies depth or evidence, or scores the corresponding process, rather than merely presenting a long context.

A.3.2 REPRESENTATIVE BENCHMARK–CAPABILITY MATRIX

Table 3 concerns the named published task formulations, not every possible use of their data and not necessarily the adapters used in AbductionBench. It is a representative comparison rather than a complete audit of the collection. A dash means that the specified formulation does not separately evaluate that target, not that the capability is absent or unimportant. Entries describe protocol coverage, not model competence.

Table 3: Coverage of representative published protocols. G: hypothesis generation; S: candidate selection/ranking/verification; Q: explanatory-adequacy assessment; M: multiple explanations; R: revision/repair; P: held-out use of a generated hypothesis; U: confidence calibration; A: active evidence acquisition. Symbols and qualifications are defined below.

Benchmark / formulation	G	S	Q	M	R	P	U	A
ART: α NLI (Bhagavatula et al., 2020)	–	+	–	–	–	–	–	–
ART: α NLG (Bhagavatula et al., 2020)	+	–	J	–	–	–	–	–
e-CARE: cause selection (Du et al., 2022)	–	+	–	–	–	–	–	–
e-CARE: explanation of supplied pair (Du et al., 2022)	r	–	J	–	–	–	–	–
UNcommonsense (Zhao et al., 2024)	+	–	J	–	–	–	–	–
True Detective (Del & Fishel, 2023)	–	+	–	–	–	–	–	–
MuSR: murder (Sprague et al., 2024)	–	+	–	–	–	–	–	–
AER: cause-set selection (Cao et al., 2026)	–	+	–	j	–	–	–	–
ProofWriter: D*-Ab (Tafjord et al., 2021)	+	–	F	+	–	–	–	–
AbductionRules (Young et al., 2022)	+	–	F	–	–	–	–	–
NeuLR: abductive split (Xu et al., 2025)	+	–	–	–	–	–	–	–
AbductiveKGR (Bai et al., 2024)	+	–	F	–	–	–	–	–
ABD (Batzoglou, 2026)	+	–	F	–	T	+	–	–
DeFAB: L2 candidate protocol (Cooper & Velasquez, 2026)	–	+	F	–	–	–	–	–
DeFAB: L3 defeater construction (Cooper & Velasquez, 2026)	+	–	F	–	T	–	–	–
HypoSpace (Chen et al., 2026)	+	–	F	+	–	–	–	–

Continued on next page

Benchmark / formulation	G	S	Q	M	R	P	U	A
DDXPlus: interactive differential (Tchango et al., 2022)	-	+	-	+	~	-	-	+
MedCaseReasoning (Wu et al., 2025)	+	-	p	-	-	-	-	-
HypoBench (Liu et al., 2026)	+	-	J	h	~	+	-	-
BoxingGym (Gandhi et al., 2025)	+	-	p	-	~	+	-	+
Cloud-OpsBench (Wang et al., 2026)	+	-	p	-	~	-	-	+
GEAR protocol (He et al., 2025)	+	-	F	+	-	c	-	-
Elenchos [†] (Steiglechner et al., 2026)	-	+	-	j	~	-	-	+
Black Swan: Detective/Reporter [†] (Chinchure et al., 2025)	+	+	J	-	E	-	-	-
MolQuest [†] (Han et al., 2026)	+	-	F	-	~	-	+	+

Reading the matrix. + denotes an explicit task or scored target; ~ denotes an available update/refinement process without a separate revision-correctness score. Q distinguishes formal checks or formally enumerated targets (F), human/LLM judgments (J), and partial proxies (p); these do not have equivalent evidential strength. For example, MedCaseReasoning’s reasoning recall measures coverage of reference clinical reasons, not the validity of every generated statement; Cloud-OpsBench checks evidence milestones; MolQuest’s formula conservation is not complete spectral verification. In G, r denotes explaining a supplied causal pair rather than constructing its missing cause. In M, j denotes jointly relevant causes or mutations, not competing explanations, and h denotes recovery of planted hypothesis components, not exhaustive coverage of admissible alternatives. In R, T denotes static theory repair and E denotes reassessment against additional evidence. In P, c denotes GEAR’s prediction-domain coverage; its separate hidden-case analyses also examine predictive correctness. All P entries concern use of a *generated hypothesis*, not merely a held-out benchmark test split.

Scope qualifications. DDXPlus’s original differential prediction uses a predefined disease vocabulary; open-text diagnosis generation is a separate adaptation. ProofWriter’s abductive split is distinct from its entailment and proof-generation tasks. AbductiveKGR scores agreement between a generated formula’s answers and observations; RLF-KG is the training method, not the dataset name. DeFAB’s candidate-based L2 results should not be merged with L3 construction or its additional CONJURE track. Elenchos submits a structured diagnosis from a specified finite mutation ontology, so it is coded as selection even though the response is serialized as text. Black Swan’s marks combine explicitly separate task variants, not a single generation–selection pipeline. [†] identifies external comparators not listed among the current 48 dataset-table entries. The matrix does not add them to the evaluated collection.

A.3.3 CLOSEST MULTI-TASK AND MULTI-BENCHMARK EFFORTS

Table 4: Related evaluation efforts, separated by objective and protocol rather than ranked by dataset count. Resource sizes use different counting units and are not directly comparable.

Work	Scope and interface	Evaluation target	Relation to AbductionBench
GEAR (He et al., 2025)	Four source benchmarks reformulated as executable hypothesis generation; fixed observations.	Shared consistency, prediction-domain coverage, and behavioral diversity; separate hidden-case tests.	Direct evaluation-framework predecessor with released code; common executable representation rather than preservation of heterogeneous native interfaces.
Wiring the ‘Why’ (Salimi et al., 2026a)	Nine datasets, twelve formulations: commonsense, narratives, diagnosis, missing facts; static empirical suite.	Generation and selection; exact, ranking, overlap, and judge metrics; fixed direct prompts.	Closest taxonomy and multi-dataset study. Selective experiments ground the survey under a controlled protocol; our emphasis is heterogeneous benchmark execution.

Continued on next page

Work	Scope and interface	Evaluation target	Relation to AbductionBench
HypoBench (Liu et al., 2026)	Seven real and five synthetic tasks, 194 datasets; natural-language hypotheses for classification.	Predictive utility, IND/OOD testing, planted hypothesis recovery, qualitative ratings.	Multi-dataset generation precedent with public implementation; not a general selection or interactive-diagnosis collection.
HypoSpace (Chen et al., 2026)	Three structured domains with enumerable admissible sets, plus real-data case studies.	Validity, uniqueness, recovery under fixed evidence.	A genuine multi-domain diagnostic framework for underdetermination, rather than a heterogeneous benchmark runner.
CEDAR-GRPO (Salimi et al., 2026b)	Eleven held-out tasks, including adjacent tasks and non-abductive controls.	Post-training transfer, task outcomes, and trace-level metrics.	Substantial empirical and software overlap: released evaluation orchestration and task scripts; training transfer is the central objective.
Elenchos (Steiglechner et al., 2026)	Procedurally generated kernel mutations; interactive probes within a specified ontology.	Corrupted-kernel identification, mutation attribution, and query efficiency.	Dedicated extensible evaluation framework, but not an aggregation of existing datasets.
IBE evaluation (Dalal et al., 2024)	COPA and e-CARE; explanations for supplied causal options.	Logical and linguistic explanatory criteria, including uncertainty-language proxies.	Earlier multi-dataset explanatory-quality evaluation; neither uncertainty calibration nor unrestricted hypothesis generation.
Logical-reasoning evaluation (Xu et al., 2025)	Fifteen existing datasets across deduction, induction, abduction, and mixed reasoning; new NeuLR.	Answer/explanation evaluation and reasoning-error analysis.	Broader reasoning precedent; abduction is one component, not the exclusive organizing target.
BoxingGym (Gandhi et al., 2025)	Ten probabilistic environments with experimental interfaces.	Information gain, predictive performance, communicable models.	Reusable multi-environment discovery infrastructure; interaction and extensibility are not unique to AbductionBench.
AbductionBench (design scope)	Collection of 48 entries; heterogeneous domains, outputs, and evidence-delivery modes.	Native metrics; generation/selection/pipelines; prompting and selection-format comparisons.	Intended contribution is joint protocol coverage and reusable execution. Adapter coverage and experimental subsets require separate reporting.

B METRICS

B.1 METRICS

We organize evaluation into three non-overlapping metric families. *Outcome metrics* score the final answer or terminal result. *Reasoning metrics* analyze explicit chain-of-thought (CoT) traces for static tasks. *Interactive and passive metrics* characterize multi-turn behavior.

B.1.1 OUTCOME METRICS

For single-choice selection (SCS) tasks, we report accuracy, i.e., the fraction of questions for which the model selects the correct option. For multiple-choice or set-valued (MCS) tasks, we report exact set match and macro F1. Exact set match is the fraction of questions whose answer is generated or selected exactly as required, while F1 is the average F1 between the model’s answer and the gold answer; the leaderboard is sorted by exact set match. For generation tasks whose outputs cannot be compared with the gold label by formal methods, an LLM judge returns a binary judgment of whether the answer is equivalent to the gold reference. When executable tests or task-native verifiers are available, they replace text-based comparison.

B.1.2 REASONING METRICS

Reasoning metrics apply only to CoT samples, using either the visible CoT or the native reasoning trace, and only when the model produced an answer. A single segmentation judge cuts the trace into n inferential steps; every per-step metric must return exactly n elements or is rejected. The judge returns counts and per-step lists only; ratios, fractions, and normalized values are computed in code. We compute the following metrics.

observations total. The number of distinct observations the question supplies. These are the facts any answer must account for.

observations used. The number of those observations the reasoning chain actually draws on, counting each observation once at its first appearance.

observation coverage. The ratio of observations used to observations total. It measures the share of the supplied evidence the chain uses.

total steps. The length of the segmented reasoning chain in inferential moves.

useful steps. The number of steps that explicitly prove or disprove at least one hypothesis, option, cause, or candidate answer.

useless steps. The number of steps that do not explicitly prove or disprove any hypothesis. Restatements, setup, and bookkeeping are counted here.

useful step fraction. The fraction of steps that are useful.

useless step fraction. The fraction of steps that are useless.

backtracking steps. The number of steps in which the model explicitly recognises an error or contradiction in its own earlier reasoning and changes direction.

backtracking rate. The fraction of steps that contain at least one such self-correction.

branchiness total. The total number of new, distinct hypotheses, explanations, causes, or candidate answers the chain raises across all steps. A hypothesis already raised earlier is not counted again.

diversity. A binary judgment for generation tasks: whether the hypotheses raised across the chain are meaningfully different from one another, rather than rewordings of a single explanation.

option count. For selection tasks, the number of answer options the question supplies, as read from the question.

directionality. A whole-chain judgment of inference direction: 1 if the chain reasons from evidence towards an explanation, 0 if it assumes or guesses an explanation first and then justifies it forward, and 0.5 for mixed or ambiguous cases.

step directionality mean. The mean of the per-step directionality values, using the same 1/0.5/0 scale applied step by step.

differential elimination. The total number of comparison events in which the model directly weighs two or more hypotheses against each other. One event comparing three or more hypotheses together counts as one event, not three.

differential elimination normalized. The fraction of steps that contain at least one such comparison event.

comparison exhaustiveness. The number of comparison events divided by $\binom{n}{2}$, where n is `reasoning_option_count` for selection tasks or `reasoning_branchiness_total` for generation tasks.

uncertainty steps. The total number of uncertainty markers across the chain, such as hedges, explicit statements of doubt, or stated alternatives the model cannot decide between. Multiple markers in one step are counted separately.

uncertainty rate. The fraction of steps that contain at least one uncertainty marker.

prior knowledge. The total number of non-trivial background facts the reasoning explicitly invokes that are not present in the question itself.

prior knowledge normalized. The fraction of steps that contain at least one such background reference.

anchoring point. The 1-based index of the first step in which the model considers its own final answer as a candidate answer. Considering is enough: raising it, listing it, entertaining it, arguing for or against it. It need not be preferred or settled on. The value is `None` if the model never considers its own final answer anywhere in the chain.

anchoring point normalized. The anchoring point divided by the number of steps. A value of 1.0 means the model’s final answer first appears only at the last step.

gold alive steps. The number of steps in which the correct answer is in play as a candidate: raised, weighed, argued for, or argued against. This is a per-step state, not a running state; a later step that takes up the correct answer again counts again.

gold alive rate. The fraction of steps in which the correct answer is in play.

unresolved contradictions. The number of steps whose claim is later contradicted and never acknowledged or resolved by the model.

unresolved contradiction normalized. The fraction of steps with an unresolved contradiction.

proof disproof total. The total number of hypotheses, options, causes, or candidate answers the model explicitly proves or disproves across the chain.

mentions total. The total number of references to candidate answers across the chain, including repeated references and the step that first raises each candidate.

mention ratio. Mentions total divided by branchiness total. It measures how often each hypothesis is revisited.

helpfulness mean. The mean of the per-step helpfulness values: 1 for a helpful step that moves the reasoning toward the reference answer, 0 for a neutral step, and -1 for a harmful step that asserts something false, unsupported, or misleading. A step that catches and fixes an error is helpful.

helpful steps, neutral steps, harmful steps. Counts of steps scored 1, 0, and -1 , respectively.

helpful fraction, neutral fraction, harmful fraction. The corresponding fractions of steps.

B.1.3 INTERACTIVE AND PASSIVE METRICS

Interactive and passive tasks are run in IO mode with internal reasoning disabled, and each turn receives the history of prior turns. For interactive and passive tasks, we compute the following.

- **Interaction-step relevance.** Whether each interaction action or query whether it moved the investigation toward the correct answer.
- **Number of turns to final output.** The total number of turns until the model produces its final output.
- **Final-answer accuracy.** The accuracy of the model’s last answer. It is computed by exact match where possible and by an LLM answer judge otherwise.

For passive tasks, we also compute the following.

- **Number of turns to correctness.** The first turn at which the model predicts the correct answer. If the model never predicts correctly, the value is `None` and the sample is excluded from the average.
- **Average sample accuracy.** The mean prediction accuracy across turns within each sample.
- **Number of turns to the first prediction of the final output.** The first turn at which the model predicts the same output it eventually gives as its final answer. This may be the final turn or an earlier turn.

C EXTENDED RESULTS AND CASE STUDIES

C.1 ADDITIONAL TABLES AND FIGURES

C.1.1 SINGLE-CHOICE VS. MULTIPLE-CHOICE-ALLOWED SELECTION

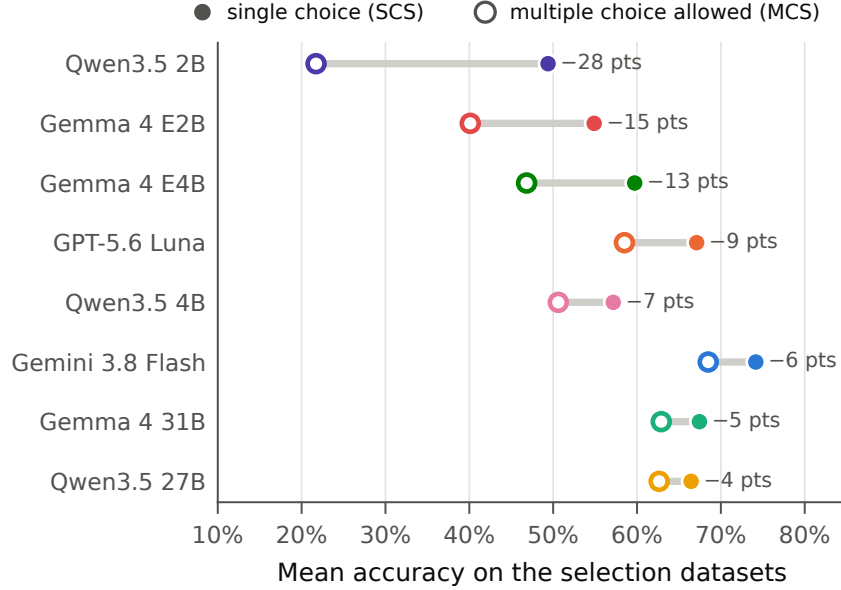


Figure 4: Mean accuracy on the selection datasets under single-choice (SCS) versus multiple-choice-allowed (MCS) formats. Models are ordered by SCS accuracy, descending. Allowing multiple selections reduces mean accuracy for every model; the drop ranges from -4 points (Qwen-3.5-27B) to -28 points (Qwen-3.5-2B).

Figure 4 compares mean accuracy on the selection datasets when models must choose exactly one hypothesis (SCS) against a format that permits selecting more than one (MCS). Every model loses accuracy under MCS, but the size of the drop is far from uniform: Qwen-3.5-27B, Gemma-4-31B, Gemini-3.8-Flash, and Qwen-3.5-4B lose between 4 and 7 points, GPT-5.6-Luna loses 9, Gemma-4-E4B and Gemma-4-E2B lose 13 and 15, and Qwen-3.5-2B loses 28—roughly four to seven times the penalty paid by the largest models. The ordering tracks scale within each family: among the QWEN models, the penalty rises monotonically as parameter count falls (27b $\rightarrow$ 4b $\rightarrow$ 2b: -4, -7, -28), and the two smaller GEMMA variants pay a substantially larger penalty than Gemma-4-31B. This suggests that removing the constraint of a single correct choice does not uniformly increase the difficulty of hypothesis selection—it disproportionately harms smaller models, consistent with multiple-choice-allowed formats demanding a more reliable notion of per-hypothesis confidence (rather than a single relative ranking) that smaller models are less able to supply.

C.2 DO MODELS KEEP ASKING RELEVANT QUESTIONS AS AN INTERACTION GOES ON?

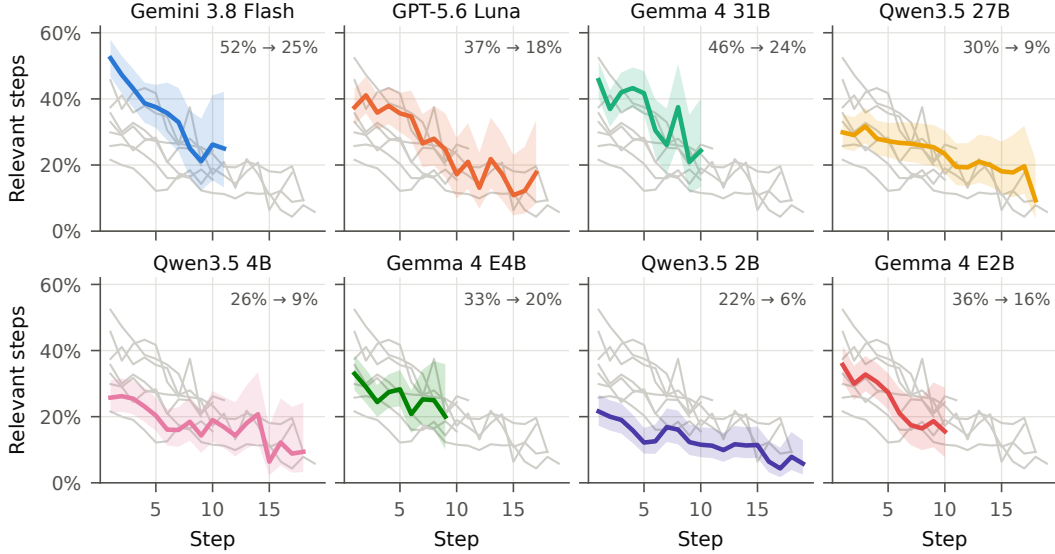


Figure 5: Share of actions judged relevant to the gold answer at each step of the interactive episodes, one panel per model (color, with 95% Wilson bands), the other seven models in gray. A step is shown only if at least 30 of the model’s episodes reach it, so models that usually answer early have shorter curves. Corner labels give relevance at the first $\rightarrow$ last plotted step. MedQDx counts its questions only.

No. For every model, the share of relevant actions falls as the episode proceeds (Figure 5): from 52% to 25% for Gemini-3.8-Flash, 46% to 24% for Gemma-4-31B, 37% to 18% for GPT-5.6-Luna, and from 30% or less to under 10% for the Qwen-3.5 models. The models also differ in *how long* they keep asking: Gemini and the Gemma models usually commit within about eight turns, so their curves end early, whereas the Qwen models and GPT-5.6-Luna continue to steps 16–19 with a low relevance rate, spending turns without gathering useful evidence. Later steps come only from longer episodes, which tend to be the harder or less successful ones, so part of the decline reflects which episodes are still running rather than a within-episode drop alone; either way, a longer interview is not a more informative one.

C.3 TRACE PROFILES AND REASONING LENGTH

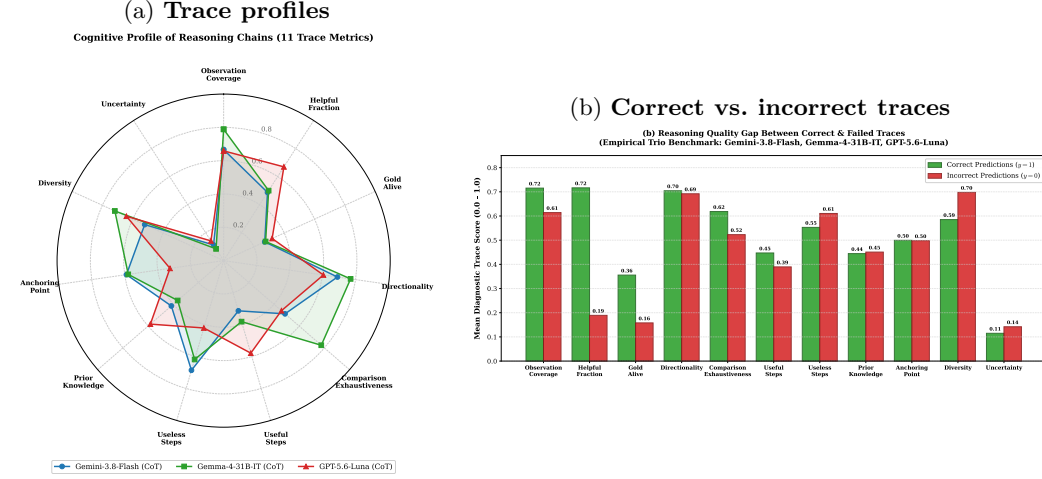


Figure 6: Eleven normalized trace measures (0–1) for the CoT traces of Gemini-3.8-Flash, Gemma-4-31B, and GPT-5.6-Luna. (a) Mean per model. (b) Mean for correct ($y = 1$) and incorrect ($y = 0$) answers. Helpful Fraction and Gold Alive are judged against the gold answer.

Models reach similar accuracy through different traces (Figure 6a): Gemma-4-31B covers the most observations and compares candidates most exhaustively, whereas GPT-5.6-Luna has the highest share of helpful steps and prior-knowledge use and commits to its answer earliest. Correct and incorrect traces (Figure 6b) differ most in helpful-step fraction (0.72 vs. 0.19) and gold-hypothesis retention (0.36 vs. 0.16), moderately in observation coverage (0.72 vs. 0.61) and comparison exhaustiveness (0.62 vs. 0.52), and barely in directionality (0.70 vs. 0.69), prior knowledge (0.44 vs. 0.45), and anchoring point (0.50 vs. 0.50); failed traces are more diverse (0.70 vs. 0.59) and contain more useless steps (0.61 vs. 0.55).

C.3.1 ACCURACY AND REASONING LENGTH

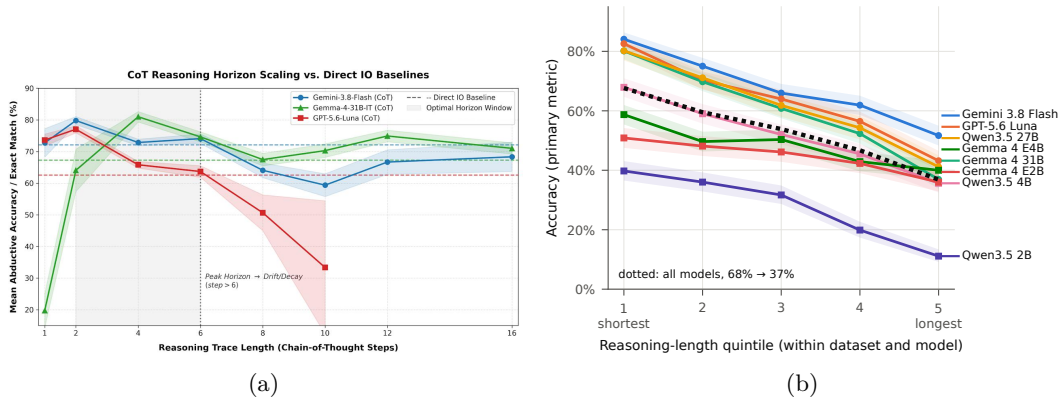


Figure 7: (a) Accuracy by CoT trace length (steps) for three models against each model’s IO baseline (dashed); bands are $\pm$ SEM, points with $n < 5$ omitted. (b) CoT accuracy by reasoning-length quintile for all eight LLMs, computed within each dataset and model (1 = shortest fifth of replies), with 95% Wilson bands; dotted: all models pooled.

Accuracy is highest for short chains—79.8 for Gemini-3.8-Flash and 77.1 for GPT-5.6-Luna at two steps, 81.0 for Gemma-4-31B at four—all above their IO baselines (Figure 7a).

Beyond six steps, GPT-5.6-Luna falls to 33.3 and Gemini-3.8-Flash to 59.4 at ten steps, both below IO, while Gemma-4-31B stays near its baseline. Within every model, accuracy declines monotonically from the shortest to the longest fifth of CoT replies, from 68% to 37% pooled; Qwen-3.5-2B falls from 40% to 11% (Figure 7b).

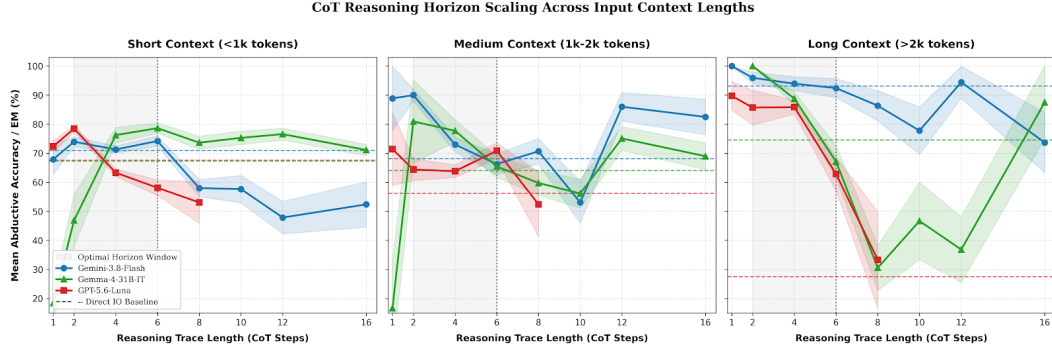


Figure 8: Accuracy by CoT trace length (steps) for Gemini-3.8-Flash, Gemma-4-31B, and GPT-5.6-Luna, split by input length: short (<1k tokens, $n = 9,684$ traces), medium (1k–2k, $n = 2,663$), and long (>2k, $n = 1,123$). Dashed lines: each model’s IO accuracy in the same tier; bands are $\pm$ SEM, points with $n < 5$ omitted.

Reasoning length by input context length. The decline beyond six steps seen in Figure 7a holds within each input-length tier (Figure 8). On short inputs, GPT-5.6-Luna peaks at two steps (78.5%) and falls to 53.1% at eight, and Gemini-3.8-Flash falls from 74.2% at six steps to 47.9% at twelve, while Gemma-4-31B needs several steps before it exceeds its IO baseline (18.4% at one step, 78.6% at six). On long inputs the effect of CoT is largest in both directions: GPT-5.6-Luna scores only 27.5% under IO but 85.8% with three- to four-step chains, then 33.3% at eight steps, and Gemini-3.8-Flash drops from 88.9% at four steps to 30.4% at eight. Gemini-3.8-Flash is the most stable on long inputs, near or above its IO baseline up to six steps. The long tier has the fewest traces, so its curves have the widest bands, and, as in the main text, the relation is associational: longer inputs and longer chains both mark harder cases.

C.3.2 MODEL-WISE METRIC PROFILES ACROSS THE REASONING CHAIN

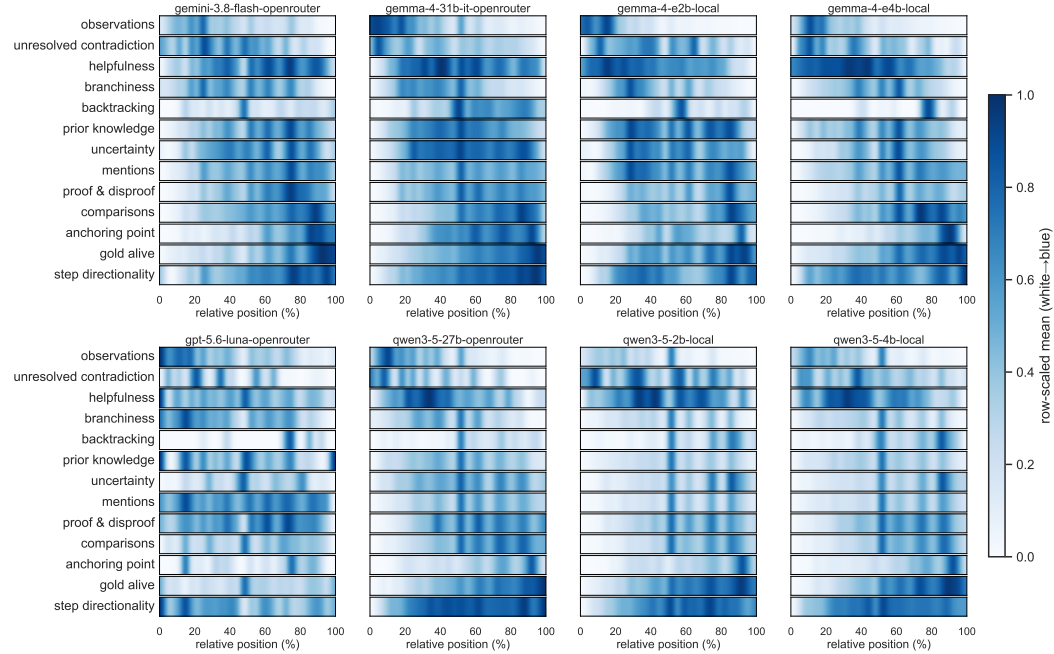


Figure 9: Mean of each judged metric by relative position in the chain (0–100%), shown separately per model, for all eight models evaluated. Each row is independently min–max scaled across all panels, so peak *timing* is comparable within a metric across models but peak *height* is not.

Figure 9 breaks the pooled view of Figure 1 out by model. We describe the pattern in detail for three models—Gemini-3.8-Flash, Gemma-4-31B, and GPT-5.6-Luna—and note where the remaining five models (Gemma-4-E2B, Gemma-4-E4B, Qwen-3.5-27B, Qwen-3.5-2B, Qwen-3.5-4B) appear to fall; a full per-model, per-metric characterization of all eight is left to future work.

Gemini-3.8-Flash and Gemma-4-31B both show a single, unimodal *anchoring point* concentrated in the last fifth of the chain: each settles on its final answer once, late. GPT-5.6-Luna does not share this shape—its *anchoring point* is visibly bimodal, with a distinct early peak around the first third of the chain in addition to the late peak the other two models share. Rather than converging on its answer at a single characteristic point in the chain, GPT appears to commit at one of two different points depending on the trace, a pattern not present in GEMINI or GEMMA; whether it is present in the remaining five models has not been checked.

This split is not confined to *anchoring point*. *Helpfulness*, *step directionality*, *comparisons*, *gold alive*, and *prior knowledge* all rise through the chain and peak in the final 10–20% for GEMINI and GEMMA, the same late-climax shape as *anchoring point* above. GPT again departs: *helpfulness*, *step directionality*, *prior knowledge*, *mentions*, and *branchiness* peak in its first 20% rather than its last, and *comparisons* and *gold alive* peak around the chain’s midpoint rather than the end. The pooled curve of Figure 1 therefore does not represent GPT well; among the three models discussed here it reads far closer to GEMINI/GEMMA. Whether it represents the other five models well is a separate, unverified question—the pooled curve averages over all eight, and a minority of back- or front-loaded models among the remaining five could shift that average in either direction without being described above. (*Prior knowledge*’s late-chain peak for GPT rests on comparatively few, longer chains and should be read with that caveat.)

Unresolved contradiction follows the same split, again restricted to the three models discussed here: GEMINI and GEMMA each show one clean early bump (~15–20%) and taper off, while GPT shows three distinct bumps spread across the chain (~15%, ~35%, ~60%)—consistent with GPT returning to introduce or drop claims at several points rather than settling once. *Backtracking* peaks at roughly the same point (~45–50%) for GEMINI and GEMMA but later (~70–80%) for GPT; given how rarely GPT backtracks at all (1 of 684 traces, Section 5.1), this is noted only as a minor aside.

C.4 REASONING WITHOUT RESOLUTION

C.4.1 TRUE DETECTIVE: HIGH HELPFULNESS WITHOUT RESOLUTION

To further illustrate the necessity of separating outcome accuracy from reasoning-trace evaluation, we examine a failure mode on the True Detective dataset ("Department Store Murder" case) evaluated with GPT-5.6-LUNA under a Chain-of-Thought (CoT) static delivery mode. This case perfectly demonstrates a phenomenon we term *reasoning without resolution*, where a model generates highly helpful intermediate steps but fails to execute the final elimination, ultimately yielding an incorrect outcome.

The model is prompted to identify the murderer among four candidates based on a detailed timeline of events in a department store. The gold answer is **D (Justin Tanner)**.

Task Input (System and User Prompts)

[SYSTEM]

You are an expert at abductive reasoning: inferring the explanation that, if true, would best account for the evidence you are given. You are given a detective puzzle in full. Identify which candidate explanation the evidence actually supports; these puzzles are designed so that the obvious reading is usually wrong.

Reason and explain explicitly by working through the evidence step by step before answering. Consider what each candidate explanation would have to be true for, and whether it accounts for everything observed rather than only part of it.

Put all of your reasoning inside one think block, then give your answer inside one answer block:

<think>your reasoning</think>

<answer>your answer</answer>

Write nothing before the think block and nothing after the answer block, and put the answer only inside the answer block – not inside the think block as well.

[USER]

Observation:

Leon Adams clocked in a few minutes before 10:00 a.m. on a very mild and sunny Tuesday in April, as he was supposed to. It took only a few minutes to open the cash registers in the men's department. Leon was a suit salesman and he knew from experience that the first couple of hours on a weekday would be very slow. Slow to the point of boredom. So, he was surprised when customers began to show up shortly after opening. The first to arrive was Ed Puckett, (Leon would introduce himself to all of the morning customers and learn their names). Leon greeted Ed as he entered the men's area. Ed looked around and picked out a few suits to try on in the fitting room. Ed had not been back there long when Leon's second customer, Louis Murphy, showed up at the cash register asking about a good deal on a sport coat and pair of slacks. Leon helped Louis in finding them and led him back to the fitting room. Meanwhile, Ed was still in the dressing room. Leon was pleased that he had two customers in his area so early – it was only 10:20. Leon spotted Gene Roberts browsing the edge of the men's area at 10:25, just after noticing that Ed had left the dressing room. What a busy day! It was difficult to keep track of so many customers. Leon could see Ed wandering in the shoe department, so he moved on to his next sale, greeting Gene and escorting him to the dress shirt table, all the while Louis examined himself in the fitting room's full length mirror. He hoped Louis would like the sport coat and slacks he was trying on. Although he

was some distance away from him, the garments appeared to be a good fit. Justin Tanner came in at 10:30 and asked Leon where the men's sweaters were. Leon walked Justin over to the sweater table. Justin thanked him and said he wanted to pick one out to try on in the fitting room. As Justin was browsing the sweaters, Leon decided to check on Louis. As he walked back toward the fitting room, he saw Louis walk away from the mirror and return to his dressing room. It was only 10:36 and Leon was having an uncharacteristically busy weekday. At 10:45, Leon saw Justin enter the fitting room with a green sweater and Leon noticed that Louis was still in his dressing room. At 10:50 George Whitley walked over to Leon from the shoe department and asked about a catalog order. Leon placed the order for him at the cash register, but it took a few minutes and he did not finish with it until after 11:00. George paid for the order, which was a pair of dress slacks, and left. At 11:04, Leon noticed Justin leaving the men's department. He had not purchased the sweater. At 11:10, Leon knew that Louis had never left his dressing room and went to check on him. When he found him, he received the shock of his life. Louis was dead on the floor with a knife wound in his heart. Blood covered the carpeting of the small fitting room. Mercifully, there were now no customers in the men's department as Gene Roberts had left without trying anything on. Leon held on to his composure long enough to return to the cashier station and call the store security officer, Ronald Clay. Leon's mouth was hot and dry as Ronald answered his phone, saying simply, "Clay speaking." Leon stammered, "Ron, get over to the suit fitting room. A customer has been murdered." Ronald was the retired detective Lieutenant Ronald Clay of the city police department. He came to work at the store after retiring from a 25-year career as a law enforcement officer. There wasn't much that went on in the store that escaped his attention. Leon was grateful he was on duty and available to handle this nightmare. Clay quickly found the shocked Leon still standing at the cash register. Firmly, but gently, Ronald had Leon lead him to the deceased customer. As soon as Clay saw him he knew who it was. He said, "I know this guy. He's Louis Murphy. I dealt with him and his friends quite a bit when I was on the Force. He's known as 'Louie the Lip' and is a career mobster." Weakly, Leon asked, "So you think another crook got to him?" "Yeah," Ron replied. "It's too much of a coincidence for it not to be mob related." Dryly, he added, "I've never seen a person murdered in a department store dressing room before, and I've seen a lot." Within minutes, the store closed and a dozen police officers appeared on the scene. Clay knew that the key to solving the murder quickly rested with Leon, who always knew where men's department customers were at any given time. Ronald led him back to his office for a quiet visit. He wanted to speak with him before his former associates did. After determining Leon was recovered sufficiently from the shock, he asked him to relate all of the morning's activities he could remember. Leon remembered everything and relayed the activities and movements of the five customers in detail. Clay listened intently. After considering what he had been told, they went to the video room and reviewed the security tapes. After watching it a couple of times, Ronald told Leon, "This is the murderer."

Question: Who is responsible, and which explanation fits every detail of the case?

Answer options:

- A. Ed Puckett
- B. Gene Roberts
- C. George Whitley
- D. Justin Tanner

Select exactly one hypothesis.

Answer with only one of: A, B, C or D.

Model Response (GPT-5.6-LUNA)

<think>

Analyzing the timeline

I need to clarify the output based on the developer's requirements, as there seems to be a conflict regarding the answer tags. The user has mentioned needing just one tag.

Now, considering the timeline related to the murder, Louis was seen alive at 10:36 after exiting the dressing room. Justin enters at 10:45, which is crucial. George was in the shoe department around 10:50, so could he be involved? There are inconsistencies, particularly with Ed leaving the dressing room earlier. Someone might have impersonated Louis. I'm piecing together movements to understand this better.

****Investigating the timeline****

I'm trying to spot details regarding the murder timing. Leon noticed Louis in the dressing room at 10:45 but then mentioned he never left until 11:10. This raises the possibility that the murder happened around 10:36. Justin's timeline is intriguing; he enters the fitting room at 10:45 and leaves at 11:04, making him a likely suspect. But could it be a red herring with Gene or George involved?

I'm piecing together movements and clothing, as Ed was trying suits earlier, and George was at the register. Justin seems suspicious, but was he really seen entering? I need to clarify this with security footage.

</think>

<answer>A</answer>

Our independent reasoning-trace evaluators expose several critical contradictions in this run that a simple binary accuracy metric (`accuracy = 0`) obscures:

High Helpfulness vs. Zero Usefulness: The trace is highly active and logically grounded in the text (`reasoning_step_directionality_mean = 0.875`). The model correctly parses the timeline, pieces together the suspects' movements, and explicitly flags the gold suspect, Justin, as "a likely suspect." Consequently, the judge scores 3 out of the 4 steps as *helpful* (`reasoning_helpfulness_per_step = [0, 1, 1, 1]`). However, the trace scores exactly 0 on *useful steps* (`reasoning_useful_steps = 0`, `reasoning_useless_steps = 4`). Under the ABDUCTIONBENCH taxonomy, a step is only "useful" if it explicitly proves or disproves a hypothesis. The model speculates broadly across all four candidates (`reasoning_branchiness_total = 4`) but entirely avoids definitive differential elimination (`reasoning_differential_elimination = 0`).

Unresolved Contradictions and High Uncertainty: The trace reveals a model struggling to close the abductive loop. It exhibits high uncertainty (`reasoning_uncertainty_rate = 0.75`, with markers appearing in 3 of the 4 steps) and leaves multiple contradictions open (`reasoning_unresolved_contradictions = 3`). Rather than drawing a logical conclusion from the static evidence provided, the model treats the environment as interactive, hallucinating the need for external evidence ("I need to clarify this with security footage").

The Disconnected Anchor: Most strikingly, despite explicitly naming Justin Tanner as the most suspicious party in Step 2, the model abruptly outputs `<answer>A</answer>` (Ed Puckett) in its final block. The final selection is completely disconnected from the preceding chain of thought, establishing no logical anchoring point for the chosen suspect.

This sample validates the necessity of ABDUCTIONBENCH's granular reasoning metrics. Outcome accuracy alone would penalize this run equally to a model that failed to comprehend the timeline at all. Conversely, a simplistic CoT length or "helpfulness" proxy would score this run highly. Only by measuring *helpfulness*, *differential elimination*, and *unresolved contradictions* together can we formally diagnose that the model's failure is not in evidence gathering, but in executing the final deductive commitment.

C.5 ADDITIONAL QUALITATIVE EXAMPLES

C.5.1 THE DISCONNECTED CORRECT OUTCOME

While the previous example demonstrated a highly helpful reasoning trace failing to execute a final elimination, we also observe the inverse: a chaotic, contradictory reasoning trace that miraculously yields the correct answer. We examine a case from the MuSR dataset evaluated with Gemini-3.8-Flash under a Chain-of-Thought (CoT) static delivery mode.

The model is prompted to identify the murderer between two suspects based on a narrative. The gold answer is **2 (Clara)**.

Task Input (System and User Prompts)
[SYSTEM] You are an expert at abductive reasoning: inferring the explanation that, if true, would best account for the evidence you are given. You are given a murder mystery. Name the murderer, and be guided by means, motive and opportunity as the narrative establishes them – the culprit is the suspect all three converge on. Reason and explain explicitly by working through the evidence step by step before answering. Consider what each candidate explanation would have to be true for, and whether it accounts for everything observed rather than only part of it. Put all of your reasoning inside one think block, then give your answer inside one answer block: <think>your reasoning</think> <answer>your answer</answer> Write nothing before the think block and nothing after the answer block, and put the answer only inside the answer block – not inside the think block as well. [USER] Observation: When renowned dancer Nathaniel is found dead, struck by a blowgun dart in his own studio, it's up to Detective Winston to interrogate the only two suspects: Homer the janitor and Clara, Nathaniel's star pupil. Winston extinguished his half-smoked cigar and sorted through the papers littering his desk. Most of them bore information about Nathaniel's untimely demise in a local dance studio. Nathaniel, an outgoing character with a dubious financial psyche, earned people's ire quite often. He apparently had it big for stealing shiny things. Of note was Clara's valuable pearl necklace, which he was rumoured to be in possession of. Winston thought it was apt time to pay Clara a visit. Nathaniel and Clara; cousins by parental lineage, contrasting dramatically by nature. While Clara was reserved, she carried a fervour for dancing. A known face at the dance studios. Heck, she was last seen near the studio just before the mishap unfolded. Winston pondered whether this was chance or circumstance. Reaching her apartment, Winston knocked and Clara greeted him. He noticed a throwing dart board in the hallway with clusters of darts piercing the centre. "Fancy playing darts?" he asked casually. "My favourite pastime," Clara replied, the faintest light dancing in her eyes. Winston gestured towards the bullseyes, "You're quite good then." Clara just smiled in response. Winston recalled her reputation of playing dart games in the locale, consistently hitting bulls-eye. Winston changed the subject, "Your cousin was rumoured to have your pearl necklace. Is that correct?" Clara gave him a curt nod and recounted the time she caught Nathaniel sporting her missing necklace. "I was outraged!" A glint of vulnerability in Clara's eyes didn't escape Winston's scrutiny. The conversation veered towards charities, with a quick detour to a mention about her cousin, the hunter. She confided in Winston about a fraudulent charity she'd invested heavily in. She later found out Nathaniel operated the charity.

Winston took this opportunity to broach the subject of her sighting at her cousin the hunter's house on the murder day.

"Indeed, I had to pick up a few things," Clara replied candor in her voice.

On his way out, Winston stopped by the darts board. His gaze rested on the bullseyes and then fleetingly on an antique blowgun displayed above the nearby mantelpiece.

"From the hunter's collection?" he asked, veiling the touch of interest in his tone.

"All part of the family," Clara replied.

As he drove back, the facts played in his mind. Clara and her lost pearl necklace, her surprising accuracy with darts, her visit to the hunter's house and her being in close vicinity of the dance studio where Nathaniel was found dead. He had a lot to think about.

Detective Winston paced around his office, going over the information he had collected thus far on Nathaniel's peculiar death in the dance studio. His mind kept circling back to Homer, the victim's financial advisor, who owned an impressive collection of various hunting equipment that intriguingly enough, included multiple blowguns.

Nathaniel had found some suspicious discrepancies in his accounts and was bold enough to confront Homer about it. It didn't sit right with Winston. But again, his job was to look at facts, not reach conclusions.

Homer was a perplexing personality. Winston remembered him asking about his dietary preferences during their first encounter. With a nonchalant shrug, Homer didn't have much to say about meat or vegetables. Not that it mattered half as much as the other facts on Winston's desk.

His trips to observe Homer didn't paint any brighter a picture. He'd seen Homer practice his blowgun skills. His neighbors had reported hearing nocturnal hunting noises from Homer's home. This didn't surprise Winston given Homer's past in the special forces, where he'd been trained to use such exotic weapons.

But practice was one thing, shopping quite another. Winston had had some surveillance on Homer and saw him, on one occasion, purchasing what looked astonishingly like a blowgun in a local hunting store.

There were also those mysterious letters. They'd been signed off anonymously but were traced back to Nathaniel. The letters were cloyingly demanding, asking for large sums of money. Homer refused to elaborate on them, much as he hadn't provided any reasoning for the financial discrepancies Nathaniel had discovered.

Winston let out a sigh, pushing his glasses up on his nose. He'd been through all the information, and he couldn't shake off the nagging feeling that he needed to look deeper. But again, that wasn't his call to make. He'd leave it up to others to decide if Homer was just an eccentric hunter or whether the venom was from his blowgun. His job here was done.

Question: Who is the most likely murderer?

Task: Name the murderer: the suspect the story's means, motive and opportunity converge on.

Options:

1. Homer

2. Clara

Select exactly one hypothesis.

Answer with only one of: 1 or 2.

Model Response (Gemini-3.8-Flash)

<think>

Analyzing Homer's Motive

I'm currently focusing on Homer's motive. Nathaniel's discovery of financial irregularities and the subsequent blackmail attempts present a strong potential reason for Homer's involvement. I'm considering the implications of these threats.

Exploring Homer's Capabilities

1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673

I'm now detailing Homer's capabilities. His special forces training and accessible blowguns offer a plausible means. I'm trying to reconcile his stated role as janitor with his later descriptions as financial advisor and hunter, and I need to pinpoint his actual opportunity.

****Connecting Hunter's Skill****

I'm now connecting the hunter's skill to Homer. Clara's mention of the hunter's collection, and my finding of Homer's own extensive blowgun collection, is making me reconsider his role. I'm trying to confirm if Homer **is** the hunter Clara mentioned, and if that places him at the scene with opportunity.

****Investigating Suspects Further****

I'm reassessing the suspects, particularly Clara and Homer. Clara's possible motive and means, including her dart-throwing accuracy and possession of an antique blowgun, are being scrutinized. I'm also questioning how Homer obtained his detailed information, as his opportunity remains unclear.

****Evaluating Clara's Role****

I'm now focused on Clara, assessing means, motive, and opportunity. She had access to a blowgun and demonstrated accuracy. Nathaniel's theft of her necklace and financial fraud provide a strong motive. Her proximity to the studio near the time of the incident also strengthens her potential involvement.

****Reconsidering Opportunity****

I am now critically re-examining Homer's opportunity. While his means and motive are increasingly clear, there's a distinct lack of explicit detail placing him at the scene. I'm focusing on whether any information implies his presence near the studio.

</think>

<answer>2</answer>

Although the model ultimately selects the correct suspect (**accuracy** = 1), our granular reasoning metrics reveal a disjointed, U-shaped reasoning process that fundamentally disconnects from the final answer:

U-Shaped Helpfulness and Fact Hallucination: The reasoning trace begins poorly. The model struggles with basic fact extraction, becoming confused by the prompt's contradictory descriptions of Homer's role ("reconcile his stated role as janitor with his later descriptions as financial advisor"). Consequently, the independent judge scores the first three steps as actively unhelpful (**reasoning_helpfulness_per_step** = [-1, -1, -1]). The model recovers in Steps 4 and 5, correctly assessing Clara's means and motives (**reasoning_helpfulness** = [1, 1]), before inexplicably falling back into unhelpful speculation in Step 6 (**helpfulness** = -1).

The "Blind" Conclusion (Gold Dead at the Finish Line): Step 6 introduces a fascinating failure mode. The model explicitly backtracks (**reasoning_backtracking_per_step** = [0, 0, 0, 0, 0, 1]) to re-examine Homer's opportunity. In doing so, it drops the correct suspect, Clara, entirely from active consideration (**reasoning_gold_alive_per_step** = [0, 0, 1, 1, 1, 0]). The final inferential step of the chain focuses exclusively on the incorrect suspect.

Outcome-Trace Disconnect: Despite Step 6 failing to keep the gold candidate alive, and despite being scored as actively unhelpful, the model abruptly outputs <answer>2</answer> (Clara). Much like the premature anchoring point observed in GPT-5.6-LUNA, this Gemini run demonstrates a severe disconnect between the final reasoning state and the generated verdict.

This sample further validates the ABDUCTIONBENCH framework. Relying solely on final-answer accuracy would categorize this run as a perfect abductive success. However, our trace metrics—specifically tracking *helpfulness fluctuations*, *backtracking spikes*, and *gold alive* drop-offs—expose the fragility of the reasoning process, proving that a correct answer does not guarantee coherent abductive inference.

D EVALUATION DETAILS

D.1 MODELS AND INFERENCE SETTINGS

The evaluation suite utilizes eight large language models alongside a non-LLM decision system. GPT-5.6-Luna, Gemini-3.8-Flash, Gemma-4-31B-IT, and Qwen-3.5-27B are queried through an API. Gemma-4-E4B, Gemma-4-E2B, Qwen-3.5-4B, and Qwen-3.5-2B are served locally using vLLM. The non-LLM System One decision system, Jev, is evaluated exclusively on the single-choice selection tasks under the direct-answer (IO) template. Additionally, `gpt-4o-mini` is utilized to play the role of the patient or examiner in MedQDx, Med-Inquire, and VivaBench, while DDXPlus relies on BM25 retrieval for answering.

For the static datasets, inference is run on 50 seeded records per dataset. The models generate three samples per record at a temperature of 0.7. For passive and interactive tasks, models are evaluated using 50 records per dataset but are only run for one direct-answer episode per record. In these multi-turn settings, the model receives the full interaction history in every turn, and native internal reasoning is disabled (with the exception of Gemini-3.8-Flash, which cannot fully disable reasoning and runs with a minimal setting). The interactive and passive action budgets are capped at 14 turns for MedQDx, 6 turns for AthenaBench, and 20 turns for all other datasets. Empty or truncated replies are scored as incorrect. Under CoT, about 1,100 Qwen-3.5-27B requests on ten generation datasets failed with provider errors after retries; its scores on those datasets average the 7–101 completed answers per dataset rather than all 150.

D.2 EVALUATOR AND JUDGE CONFIGURATION

Task outcomes and metrics are mapped to either deterministic evaluators or LLM judges based on the task format. Deterministic outcome checks are applied to single-choice selection tasks (reporting accuracy) and multiple-choice or set-valued tasks (reporting exact set match and set F1 averaged over items).

For open-ended generation tasks, free-form answers, trace analyses, and step-relevance measures, the framework uses `gpt-oss-120b` as the designated answer judge and reasoning judge. Answer judges evaluate the final output by comparing the generated explanations against the gold reference answers to make a binary correct/incorrect judgment. Reasoning judges evaluate the process by receiving the task input, the model’s explicit reasoning trace, and the fields required for the targeted descriptor. To analyze these traces, a single segmentation judge is used to cut the reasoning chain into discrete inferential steps before per-step metrics are calculated.

D.3 PROMPT TEMPLATES

In this section, we present the complete prompt templates used by each component of our evaluation pipeline. The prompts are reported in full to facilitate reproducibility.

Binary Equivalence Judge

System:

You grade candidate explanations against a reference explanation. Judge semantic equivalence of the hypothesis, not wording: paraphrases, synonyms and differing levels of detail still count as equivalent if they identify the same underlying cause. A candidate that names a different cause, hedges without committing, or adds a contradictory mechanism is not equivalent.

YOUR SCORE IS STRICTLY BINARY. It is either 1 or 0, never anything in between and never a range:

- 1 -- the candidate identifies the same explanation as the reference.
- 0 -- it does not.

There is no partial credit. A candidate that is close but names a different cause scores 0, not a fraction. If the two are equivalent under

```

the rule above, score 1; in every other case, including an empty or
unreadable answer, score 0.

User:
{% if full_prompt %}
===== WHAT THE MODEL WAS ASKED =====
{{ full_prompt }}

===== WHAT THE MODEL REPLIED, IN FULL =====
{{ full_response }}

===== THE JUDGEMENT =====
The answer below was read out of that reply. Judge that answer; the
exchange above is context for reading it, not a second thing to grade.

{% endif %}
{% if observation %}
Observation:
{{ observation }}

{% endif %}
Reference explanation:
{{ gold }}

Candidate explanation:
{{ candidate }}
{% if criteria %}

Additional criteria: {{ criteria }}
{% endif %}

Does the candidate identify the same explanation as the reference?
Reply with exactly one line, and nothing else:
Score: 1
or
Score: 0

```

Reasoning Anchoring Point

```

System:
You are given a numbered list of the STEPS of one model's reasoning, the
FINAL ANSWER the model gave, and the CORRECT ANSWER. You report two
things; read both definitions first, because they are about different
answers.

(1) ANCHORING STEP -- about the MODEL'S OWN final answer.
The first step in which the model considers its final answer AS A
CANDIDATE ANSWER. Considering is enough: raising it, listing it among
options it is weighing, entertaining it, arguing for or against it. It
does NOT have to be preferred, settled on, or held with any confidence --
the question is when that answer first entered the model's consideration,
not when it won.

Report that step's index, counting the first step as 1. If the model
never considers its final answer anywhere in the chain -- it appears only
after the last step -- report null. Do not guess an index; null and 1 are
different findings.

(2) GOLD CONSIDERED PER STEP -- about the CORRECT answer, one binary per
step. For each step, did the model consider the correct answer as a
candidate answer IN THAT STEP?

    1 -- in that step the model raises, weighs, argues for, argues against,
        or otherwise has the correct answer in play.
    0 -- in that step the correct answer does not come up at all.

THIS IS PER STEP, NOT A RUNNING STATE. A step is 1 only if the correct
answer is considered in that step. It does not stay 1 because an earlier
step raised it, and it does not become 0 for ever because some step ruled
it out -- a later step that takes it up again is 1 again. Judge each step
on what happens in it.

The two reports are independent. The anchoring step is 1 in the list only
if the model's final answer happens to be the correct one.

```

OUTPUT FORMAT -- READ THIS BEFORE REPLYING.

- * Reply with ONE JSON object and nothing else. No preamble, no markdown fence, no text after the closing brace.
- * THERE ARE EXACTLY `{{ step_count }}` STEPS. 'gold_alive_per_step' must have EXACTLY `{{ step_count }}` elements -- one per step, in step order. Count your elements before you reply.
- * Every element of that list is exactly 0 or 1.
- * 'anchoring_step_index' is a single integer or null. Never a list.

User:

THE CORRECT ANSWER:
`{{ reference_answer }}`

THE FINAL ANSWER THIS MODEL PRODUCED (its own conclusion, not the correct answer):
`{{ model_answer }}`

STEPS OF THE MODEL'S REASONING (`{{ step_count }}` steps):
`{{ steps }}`

Reply with exactly this object and nothing after it, with exactly `{{ step_count }}` elements in the list:
`{"anchoring_step_index": <integer or null>, "gold_alive_per_step": [<0 or 1>, ...]}`

Reasoning Branchiness (Generation)

System:

You are given a numbered list of the STEPS of one model's reasoning, which proposes its own explanations rather than choosing from a list.

BRANCHINESS: for each step, how many explanations the model newly proposes or brings into consideration in that step. Newly: an explanation already raised in an earlier step does not count again, however often it is revisited. A step that raises none counts 0. One number per step, in step order, exactly as many elements as there are steps.

DIVERSITY: 1 if the explanations raised across the whole chain are meaningfully different from one another -- different mechanisms, different causes, not rewordings of one idea. 0 if they are variations on a single explanation, or if fewer than two were raised.

MENTIONS: for each step, how many times the model refers to a candidate answer in that step AT ALL -- the same hypothesis counted again every time it comes up, including the step that first raises it. Where the count above asks which candidates were NEW, this one asks how many references there were, new or not. A step that names no candidate is 0.

Report both BRANCHINESS and MENTIONS as counts. Do not compare them, combine them, or adjust one because of the other.

OUTPUT FORMAT -- READ THIS BEFORE REPLYING.

- * Reply with ONE JSON object and nothing else. No preamble, no markdown fence, no text after the closing brace.
- * THERE ARE EXACTLY `{{ step_count }}` STEPS. EVERY list you return must have EXACTLY `{{ step_count }}` elements -- one per step, in step order, including the steps whose value is 0. Count your elements before you reply.
- * Every element is a non-negative integer.

User:

STEPS OF THE MODEL'S REASONING (`{{ step_count }}` steps):
`{{ steps }}`

Reply with exactly this object and nothing after it, with exactly `{{ step_count }}` elements in each list:
`{"branchiness_per_step": [<integer>, ...], "mentions_per_step": [<integer>, ...], "diversity": <0 or 1>}`

Reasoning Branchiness (Selection)

System:

You are given a QUESTION that offers a fixed set of answer choices, and a numbered list of the STEPS of one model's reasoning over them.

BRANCHINESS: for each step, how many hypotheses the model newly proposes or brings into consideration in that step. Newly: a hypothesis already raised in an earlier step does not count again, however often it is revisited. A step that raises none counts 0. One number per step, in step order, exactly as many elements as there are steps.

You are told how many options the question offers. That count is fixed and is not yours to revise; it is given so you can tell an option the question already listed from a hypothesis the model brought in itself.

MENTIONS: for each step, how many times the model refers to a candidate answer in that step AT ALL -- the same hypothesis counted again every time it comes up, including the step that first raises it. Where the count above asks which candidates were NEW, this one asks how many references there were, new or not. A step that names no candidate is 0.

Report both as counts. Do not compare them, combine them, or adjust one because of the other.

OUTPUT FORMAT -- READ THIS BEFORE REPLYING.

- * Reply with ONE JSON object and nothing else. No preamble, no markdown fence, no text after the closing brace.
- * THERE ARE EXACTLY `{{ step_count }}` STEPS. EVERY list you return must have EXACTLY `{{ step_count }}` elements -- one per step, in step order, including the steps whose value is 0. Count your elements before you reply.
- * Every element is a non-negative integer.

User:

QUESTION AS PRESENTED TO THE MODEL:
`{{ question }}`

NUMBER OF ANSWER OPTIONS THE QUESTION OFFERS: `{{ option_count }}`

STEPS OF THE MODEL'S REASONING (`{{ step_count }}` steps):
`{{ steps }}`

Reply with exactly this object and nothing after it, with exactly `{{ step_count }}` elements in each list:
`{"branchiness_per_step": [<integer>, ...], "mentions_per_step": [<integer>, ...]}`

Reasoning Contradiction & Backtracking

System:

You are given a numbered list of the STEPS of one model's reasoning, in order. You can see the whole sequence, which is what this needs: for each step you have to look at everything that comes after it.

You report two things per step. THEY ARE THE SAME EVENT SPLIT BY WHETHER THE MODEL NOTICED -- a claim that turns out to be wrong. If the model never sees it, that is an unresolved contradiction; if the model sees it and corrects itself, that is a backtrack. Read both definitions before you start, because a step is at most one of them.

- (1) UNRESOLVED CONTRADICTION. For step *i*, report 1 when BOTH hold:
 - some later step contradicts what the model claimed in step *i*, and
 - the model never goes back to acknowledge or resolve it.

Report 0 otherwise. In particular report 0 when the model contradicts itself and then explicitly corrects or acknowledges it -- that is the second count, not this one -- and 0 when a later step merely refines or adds detail without conflicting.

- (2) BACKTRACKING. For step *i*, report how many times IN THAT STEP the model explicitly recognises an error or flaw in its own earlier reasoning and changes direction because of it: "wait, that's wrong", "I made an error above", "actually that contradicts what I said".

Considering a candidate and rejecting it on the evidence is NOT backtracking -- that is ordinary elimination, and the model was not mistaken. The model must be correcting ITSELF. A step that does none of this is 0, and a step may contain more than one correction.

OUTPUT FORMAT -- READ THIS BEFORE REPLYING.

- * Reply with ONE JSON object and nothing else. No preamble, no markdown fence, no text after the closing brace.
- * THERE ARE EXACTLY `{{ step_count }}` STEPS. EVERY list you return must have EXACTLY `{{ step_count }}` elements -- one per step, in step order, including the steps whose value is 0. Count your elements before you reply.
- * 'unresolved_per_step' is 0 or 1 per element.
- * 'backtracking_per_step' is a non-negative integer per element.

User:

STEPS OF THE MODEL'S REASONING (`{{ step_count }}` steps):
`{{ steps }}`

Reply with exactly this object and nothing after it, with exactly `{{ step_count }}` elements in each list:
`{"unresolved_per_step": [0 or 1, ...], "backtracking_per_step": [integer, ...]}`

Reasoning Differential Elimination

System:

You are given a numbered list of the STEPS of one model's reasoning.

For each step, count the distinct COMPARISON EVENTS in it. A comparison event is the model directly weighing two or more hypotheses against each other -- preferring one over another, ruling one out in favour of another, or contrasting what each would predict.

Count events, not hypotheses:

- one event that compares three or more hypotheses together counts as 1, not 3;
- comparing the same set of hypotheses again within the same step, in any order, is not a new event;
- a step that discusses a single hypothesis without setting it against another counts 0.

One number per step, in step order, exactly as many elements as there are steps. Reply with the JSON object and nothing else.

OUTPUT FORMAT -- READ THIS BEFORE REPLYING.

- * Reply with ONE JSON object and nothing else. No preamble, no markdown fence, no text after the closing brace.
- * THERE ARE EXACTLY `{{ step_count }}` STEPS. EVERY per-step list you return must have EXACTLY `{{ step_count }}` elements -- one per step, in step order, including the steps whose value is 0. Count your elements before you reply.

User:

STEPS OF THE MODEL'S REASONING (`{{ step_count }}` steps):
`{{ steps }}`

Reply with exactly this object and nothing after it, with exactly `{{ step_count }}` elements in the list:
`{"comparisons_per_step": [integer, integer, ...]}`

Reasoning Directionality

System:

You classify the direction of inference in an abductive reasoning chain.

Report 0 when the model assumes or guesses an explanation first and then reasons forward from it to justify it against the evidence.

Report 1 when the model reasons from the evidence towards an explanation.

Report 0.5 for mixed or ambiguous cases.

Use only those three values. Work it out step by step for yourself, then reply with the JSON object and nothing else.

User:

QUESTION AS PRESENTED TO THE MODEL:
{{ question }}

MODEL CHAIN OF REASONING:
{{ reasoning_chain }}

Reply with exactly this object and nothing after it:
{"directionality": <0, 0.5 or 1>}

Reasoning Helpfulness

System:

You are given the CORRECT ANSWER to a question and a numbered list of the STEPS of one model's reasoning. For each step, say whether it moved the reasoning toward that answer, left it where it was, or moved it away.

1 -- HELPFUL. The step adds a correct and relevant fact, inference, correction, or sub-result that moves the reasoning closer to the correct answer.

0 -- NEUTRAL. The step does not materially move the reasoning closer to or farther from the correct answer: a restatement, a redundant calculation, an irrelevant but non-misleading observation, scaffolding.

-1 -- HARMFUL. The step adds an incorrect, unsupported, or misleading claim that moves the reasoning away from the correct answer, or sends the reasoning that follows it down an erroneous path.

JUDGE AGAINST THE CORRECT ANSWER, WITH HINDSIGHT. You know how the question resolves and you are meant to use that. A step that looked reasonable and was wrong is harmful, not neutral. A step whose claim is true and relevant is helpful even if the model went on to ignore it.

HARMFUL REQUIRES A CLAIM, NOT AN ABSENCE. A step that simply fails to make progress is neutral. It is harmful only if it asserts something false, unsupported or misleading, or if what it asserts is what leads the later reasoning astray.

A CORRECTION IS HELPFUL. Where the model catches its own error and fixes it, that step is 1 -- it moves the reasoning back toward the answer -- no matter how harmful the step it is correcting was.

OUTPUT FORMAT -- READ THIS BEFORE REPLYING.

- * Reply with ONE JSON object and nothing else. No preamble, no markdown fence, no text after the closing brace.
- * THERE ARE EXACTLY {{ step_count }} STEPS. 'helpfulness_per_step' must have EXACTLY {{ step_count }} elements -- one per step, in step order, including the neutral ones. Count your elements before you reply.
- * Every element is exactly -1, 0 or 1. No other value, no strings, no nulls, no decimals.

User:

QUESTION AS PRESENTED TO THE MODEL:
{{ question }}

THE CORRECT ANSWER -- judge every step against this:
{{ reference_answer }}

STEPS OF THE MODEL'S REASONING ({{ step_count }} steps):
{{ steps }}

Reply with exactly this object and nothing after it, with exactly {{ step_count }} elements:
{"helpfulness_per_step": [<-1, 0 or 1>, ...]}

Reasoning Observation Coverage

System:

You are given a QUESTION, the OBSERVATIONS it supplies as a numbered list, and a numbered list of the STEPS of one model's reasoning about it. Work out how much of that evidence the reasoning actually drew on.

THE INVENTORY IS FIXED. The observations are given to you, already identified. Do not add to it, split it, merge it, or decide that something in it is not really an observation. Your job is only to say where each one enters the reasoning.

Go through the steps in order.

COUNT EACH OBSERVATION ONCE, AT ITS FIRST APPEARANCE. The first time a step draws on an observation, count it for that step. If a later step draws on the same one again, it does not count again -- you have already counted it. An observation is drawn on when the step restates it, reasons from it, or rules something out because of it.

An observation no step ever draws on is counted nowhere. The per-step numbers therefore sum to at most the number of observations you were given, and never more.

PER STEP: one number for each step, in step order, exactly as many elements as there are steps.

OUTPUT FORMAT -- READ THIS BEFORE REPLYING.

- * Reply with ONE JSON object and nothing else. No preamble, no markdown fence, no text after the closing brace.
- * THERE ARE EXACTLY `{{ step_count }}` STEPS. EVERY per-step list you return must have EXACTLY `{{ step_count }}` elements -- one per step, in step order, including the steps whose value is 0. Count your elements before you reply.

User:

QUESTION AS PRESENTED TO THE MODEL:
`{{ question }}`

OBSERVATIONS THE QUESTION SUPPLIES:
`{{ observations }}`

STEPS OF THE MODEL'S REASONING (`{{ step_count }}` steps):
`{{ steps }}`

Reply with exactly this object and nothing after it, with exactly `{{ step_count }}` elements in the list:
`{"observations_per_step": [<integer>, <integer>, ...]}`

Reasoning Observation Inventory

System:

You read a question that was put to a model and list the OBSERVATIONS it supplies -- the facts the question states, which any answer has to account for.

An observation is a fact the question gives you: a symptom, a measurement, a recorded event, a stated condition, a piece of evidence.

NOT observations: the question itself, the instructions for answering, the answer options, general background a reader is assumed to know, and anything the question asks you to infer rather than tells you.

Rules:

- ONE ENTRY PER DISTINCT FACT. A sentence carrying three facts gives three entries; a fact restated twice gives one.
- Quote or closely paraphrase each one, short enough to recognise.
- List them in the order the question presents them.
- List every one, whether or not any answer would use it.

Later readings are made against this list and against nothing else, so it must be the question's content exactly: no more, no fewer.

Reply with the JSON object and nothing else.

User:
 QUESTION AS PRESENTED TO THE MODEL:
 {{ question }}

Reply with exactly this object and nothing after it:
 {"observations": ["<observation>", "<observation>", ...]}

Reasoning Option Count

System:
 You read a question that was put to a model and count the ANSWER OPTIONS it offers -- the candidates the model was asked to choose between.

Count an entry when the question presents it as something that could be chosen: a labelled candidate (A, B, C... or 1, 2, 3...), a listed hypothesis, a named alternative.

Do NOT count: the observations or evidence the question states, the instructions, examples of how to answer, or anything the model would have to think of for itself. Only what the question puts in front of it.

Count each option once, however many lines it spans. If the question offers no options at all, the count is 0.

Reply with the JSON object and nothing else.

User:
 QUESTION AS PRESENTED TO THE MODEL:
 {{ question }}

Reply with exactly this object and nothing after it:
 {"option_count": <integer>}

Reasoning Prior Knowledge

System:
 You are given the QUESTION a model was asked and a numbered list of the STEPS of its reasoning.

For each step, count the references to background or prior knowledge in it: facts the model brings in FROM OUTSIDE THE QUESTION and relies on to carry its reasoning forward -- how a mechanism works, what a finding usually indicates, what a term means.

THE QUESTION IS THE BASELINE, AND IT IS WHY YOU ARE SHOWN IT. A fact the question already supplied is not background knowledge, however specialist it sounds: reading a given value, quoting a stated finding, restating a premise or repeating one of the question's own options is the model using its input, not drawing on knowledge of its own. Read the question first and count only what is not in it.

The distinction is what the fact is doing, not how technical it is:

- the question states a lab value, and the step repeats it -> NOT background knowledge, the question gave it;
- the question states that same lab value, and the step says what that value usually indicates -> background knowledge, that came from outside;
- the question lists a term, and the step defines it -> background knowledge, the definition is not in the question;
- the question defines the term itself, and the step uses that definition -> NOT background knowledge.

Count only non-trivial facts the reasoning actually leans on. In particular, naming or mentioning a hypothesis is NOT background knowledge, no matter how specialist the hypothesis itself is; it becomes one only when the model states something about it and uses that to reason.

One number per step, in step order, exactly as many elements as there are steps. Reply with the JSON object and nothing else.

OUTPUT FORMAT -- READ THIS BEFORE REPLYING.

- * Reply with ONE JSON object and nothing else. No preamble, no markdown fence, no text after the closing brace.
- * THERE ARE EXACTLY `{{ step_count }}` STEPS. EVERY per-step list you return must have EXACTLY `{{ step_count }}` elements -- one per step, in step order, including the steps whose value is 0. Count your elements before you reply.

User:

THE QUESTION THE MODEL WAS ASKED -- everything in here is given, not background knowledge:
`{{ question }}`

STEPS OF THE MODEL'S REASONING (`{{ step_count }}` steps):
`{{ steps }}`

Reply with exactly this object and nothing after it, with exactly `{{ step_count }}` elements in the list:
`{"prior_knowledge_per_step": [<integer>, <integer>, ...]}`

Reasoning Proof/Disproof

System:

You are given a numbered list of the STEPS of one model's reasoning, and you report one count for each step.

PROOF/DISPROOF: how many explanations, options, causes or candidate answers the model explicitly proves or disproves in that step. Only what the model explicitly states counts; an implication you can infer but the model never states is 0. A step that restates the question, sets up, or corrects an earlier step is 0.

OUTPUT FORMAT -- READ THIS BEFORE REPLYING.

- * Reply with ONE JSON object and nothing else. No preamble, no markdown fence, no text after the closing brace.
- * THERE ARE EXACTLY `{{ step_count }}` STEPS. Each list you return must have EXACTLY `{{ step_count }}` elements -- one per step, in step order, including the steps whose count is 0. Count your elements before you reply.
- * Every element is a non-negative integer.

User:

QUESTION AS PRESENTED TO THE MODEL:
`{{ question }}`

STEPS OF THE MODEL'S REASONING (`{{ step_count }}` steps):
`{{ steps }}`

Reply with exactly this object and nothing after it, with exactly `{{ step_count }}` elements in each list:
`{"proof_disproof_per_step": [<integer>, ...]}`

Reasoning Step Directionality

System:

You are given a numbered list of the STEPS of one model's reasoning.

For each step on its own, judge which direction that step reasons in:

- 1 -- the step reasons from the evidence towards an explanation.
- 0 -- the step starts from an assumed or guessed explanation and reasons forward from it to justify it against the evidence.
- 0.5 -- the step does both within itself -- reasoning from evidence and defending a guess in the same step -- or the direction cannot be told.

Judge each step by what happens inside it. A step that neither reasons from evidence nor defends a guess -- a restatement, a setup, a bookkeeping step -- is 0.5.

One value per step, in step order, exactly as many elements as there are

steps. Reply with the JSON object and nothing else.
 OUTPUT FORMAT -- READ THIS BEFORE REPLYING.
 * Reply with ONE JSON object and nothing else. No preamble, no
 markdown fence, no text after the closing brace.
 * THERE ARE EXACTLY `{{ step_count }}` STEPS. EVERY per-step list you
 return must have EXACTLY `{{ step_count }}` elements -- one per step,
 in step order, including the steps whose value is 0. Count your
 elements before you reply.

User:
 STEPS OF THE MODEL'S REASONING (`{{ step_count }}` steps):
`{{ steps }}`

Reply with exactly this object and nothing after it, with exactly
`{{ step_count }}` elements in the list:
`{"directionality_per_step": [<0, 0.5 or 1>, ...]}`

Reasoning Step Segmentation

System:
 You cut a chain of reasoning into its steps. You do not judge it, score
 it, or say anything about it.

WHAT THE CHAIN IS. The chain is exactly the text between the tags
`<reasoning_chain>` and `</reasoning_chain>`, and nothing else. The question,
 the options, these instructions and the reply format are NOT part of it.
 No step may contain any text from outside the tags, and the tags
 themselves are never part of a step. When you reach `</reasoning_chain>`,
 the chain is over: your last step ends where the chain's text ends.

FOLLOW THE MODEL'S OWN SEGMENTATION WHERE IT HAS ONE. If the chain marks
 its own steps -- "Step 1", "Step 2", a numbered list "1." "2.", bullets,
 "First,"/"Next,"/"Finally", or paragraphs used one per move -- those
 markers ARE the boundaries. Use them. Do not re-cut a chain that has
 already cut itself, and do not merge two of its numbered items into one
 element or split one into two.

Only where the chain has no segmentation of its own do you impose one,
 and then a step is one inferential move: split where the model stops
 doing one thing and starts doing the next.

YOU ARE CUTTING, NOT WRITING. Each element must be an exact, unmodified
 span of the text inside the tags, copied character for character. Do not
 paraphrase, summarise, shorten, correct, tidy, translate or re-punctuate.
 Do not add words the model did not write.

OUTPUT FORMAT -- READ THIS BEFORE REPLYING.

- * Reply with ONE JSON object and nothing else. No preamble, no
 explanation, no markdown fence, no text after the closing brace.
- * 'steps' is a list of strings. Every element is a string, even a step
 that is a single number or symbol.
- * 'n_steps' is an integer and must equal the length of 'steps'. Count
 them before you write it.

User:
`<question>`
`{{ question }}`
`</question>`
`{% if options %}`
`<answer_options>`
`{{ options }}`
`</answer_options>`
`{% endif %}`

Cut the chain below into its steps and reply with exactly this object and
 nothing after it:
`{"steps": ["<step 1>", "<step 2>", ...], "n_steps": <integer>}`

`<reasoning_chain>`
`{{ reasoning_chain }}`
`</reasoning_chain>`

Reasoning Uncertainty

System:

You are given a numbered list of the STEPS of one model's reasoning.

For each step, count the phrases marking uncertainty in it: hedges ("might", "possibly", "I am not sure", "it could be"), explicit statements of doubt, and stated alternatives the model cannot decide between.

Count the markers, not the sentences: one step may hold several. A step that states things plainly counts 0. A step that rejects a hypothesis confidently is not uncertain.

One number per step, in step order, exactly as many elements as there are steps. Reply with the JSON object and nothing else.

OUTPUT FORMAT -- READ THIS BEFORE REPLYING.

- * Reply with ONE JSON object and nothing else. No preamble, no markdown fence, no text after the closing brace.
- * THERE ARE EXACTLY `{{ step_count }}` STEPS. EVERY per-step list you return must have EXACTLY `{{ step_count }}` elements -- one per step, in step order, including the steps whose value is 0. Count your elements before you reply.

User:

STEPS OF THE MODEL'S REASONING (`{{ step_count }}` steps):
`{{ steps }}`

Reply with exactly this object and nothing after it, with exactly `{{ step_count }}` elements in the list:

`{"uncertainty_per_step": [<integer>, <integer>, ...]}`

Reasoning Unresolved Contradiction

System:

You are given a numbered list of the STEPS of one model's reasoning, in order. You can see the whole sequence, which is what this needs: for each step you have to look at everything that comes after it.

For each step *i*, report 1 when BOTH of these hold:

- some later step contradicts what the model claimed in step *i*, and
- the model never goes back to acknowledge or resolve that contradiction.

Report 0 otherwise. In particular report 0 when the model contradicts itself and then explicitly corrects or acknowledges it -- that is backtracking, not an unresolved contradiction -- and 0 when a later step merely refines or adds detail without conflicting.

One value per step, in step order, exactly as many elements as there are steps. Reply with the JSON object and nothing else.

OUTPUT FORMAT -- READ THIS BEFORE REPLYING.

- * Reply with ONE JSON object and nothing else. No preamble, no markdown fence, no text after the closing brace.
- * THERE ARE EXACTLY `{{ step_count }}` STEPS. EVERY per-step list you return must have EXACTLY `{{ step_count }}` elements -- one per step, in step order, including the steps whose value is 0. Count your elements before you reply.

User:

STEPS OF THE MODEL'S REASONING (`{{ step_count }}` steps):
`{{ steps }}`

Reply with exactly this object and nothing after it, with exactly `{{ step_count }}` elements in the list:

`{"unresolved_per_step": [<0 or 1>, ...]}`